PROGRAM SUMMARY

Program title: readcon-core
CPC Library link to program files: (to be added by Technical Editor)
Developer's repository link: https://github.com/lode-org/readcon-core
Licensing provisions: MIT
Programming language: Rust, C, C++, Python, Julia, Fortran 2008
Supplementary material: specification, ABNF grammar, conformance corpus, failure-case scripts, MPI and restart tables

Nature of problem: Rare-event campaigns stop and resume from a stored geometry. The formats in common use each drop part of the resume state: a per-axis mask, a persistent atom id, the velocity time unit, binary64 coordinates, or the periodic flags. Readers of one format also disagree with each other, so the resumed search runs on a different surface.

Solution method: CON 3.1 is a versioned grammar. A reader of a known major accepts a later minor, keeps unknown keys, and rejects an unknown major. Every velocity block declares its time unit. The default writer emits the shortest decimal that parses back to the same binary64 value. A corpus of legal and illegal frames decides what a conforming reader accepts. One Rust core sits behind one C interface, and eOn 3.6.0 reads and writes CON through it. The companion store readcon-db keeps that text, indexes composition, energy and forces, recognises gzip and zstd wrappers of the same text, and ingests frames from several MPI ranks.

Additional comments including restrictions and unusual features: A frame stores geometry and its labels. The short history of a quasi-Newton or FIRE step lies outside the frame. The default memory map of readcon-db is 2 GiB.

The restart state of a saddle search: CON 3.1, a conformance corpus, and one reference reader

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

A saddle search resumes from a geometry file, so that file is the restart state. The resume needs a per-axis mask, a persistent atom id, a declared velocity time unit, binary64 coordinates with the cell and the periodic flags, and a versioned grammar. One masked Pt(100) frame loses at least one of those fields in every common format tested here. An ASE POSCAR round trip drops the adatom's height constraint, and the resumed climbing-image band finishes 70.2 meV below the uninterrupted barrier of 0.644561366 eV. Five extended XYZ readers return four answers for one file. CON 3.1 versions the text eOn writes, pairs it with a conformance corpus, and implements it once: a Rust core behind a C interface for C, C++, Python, Julia and Fortran. Twenty-five language pairs resume the band at that barrier. A warm read of 10,000 frames of 218 atoms takes 12.6 μs per frame. The reader scales across threads and across eight MPI ranks, and the companion store readcon-db keeps the CON text, indexes it, and ingests it from several ranks.

Keywords: checkpoint, restart, specification, conformance, saddle, nudged elastic band, eOn

1. Introduction

Harmonic transition-state theory builds a rate from a barrier and from the curvatures at a minimum and at a first-order saddle [1]. The prefactor is a ratio of frequencies of the free modes [2]. The rate theories that followed keep that dependence on the barrier region [3]. A mask read wrongly removes

Email address: rgoswami@ieee.org (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

a mode, so the stored geometry has to carry the mask. A coordinate that moves on restart is a different stationary point, so the stored geometry has to come back as the same binary64 values.

The searches that find those points all stop on a file. The nudged elastic band stores a chain of images tied by springs [4]. Its climbing image pushes the highest image to the saddle, which is the number the campaign reports [5]. The tangent that image uses is estimated from its neighbours, so a displaced neighbour changes the next step [6]. Energy-weighted springs change the spacing and still resume from the same images [7]. An improved initial path is itself a file of geometries [8]. The string method stores the path as a curve of configurations [9], and the simplified string does the same [10]. A growing string adds images from the endpoints, so an interrupted growth resumes from the images already placed [11]. The dimer stores two nearby geometries and rotates them onto the lowest mode [12]. Later dimer variants change the rotation and still restart from those two geometries [13, 14]. Single-ended searches differ in how they climb, and they share the stored point as the object being compared [15]. A catalogue of stationary points is a catalogue of geometries [16]. Partial Hessian diagonalization accelerates the refinement of a saddle the file already holds [17]. A geodesic step in internal coordinates speeds the optimiser and still writes the Cartesian frame [18]. An automation layer such as Sella stops and resumes that frame [19]. Aligning the climbing image with a Hessian eigenmode is a later step on the same stored band [20]. The chapter that collects these searches is a list of algorithms with one shared record, the geometry [21].

When the event is too rare for a direct trajectory, the campaign still hands a configuration from one stage to the next. Hyperdynamics adds a bias defined on the stored configuration [22]. The bond-boost form of that bias is defined on the same point [23]. The review of accelerated molecular dynamics is a review of methods that exchange a configuration [24]. Off-lattice kinetic Monte Carlo accepts an event and writes the new state for the next search [25]. The event table is keyed by that configuration [26]. Adaptive kinetic Monte Carlo builds the table as it goes, and the campaign is the sequence of stored states [27]. Absorbing Markov chains for slow dynamics resume from the last configuration [28]. A parallel replica copies one state onto many ranks, so every replica has to start from the same geometry [29]. Dephasing starts from that same stored state [30]. Parallel trajectory splicing passes states between segments [31]. Basin hopping records the minimum it just accepted [32]. Minima hopping escapes by a short trajectory and then relaxes,

so it writes both velocities and a geometry [33].

eOn is the program that runs these searches and stops them on a file [34]. The concept record of that program is separate from any one release [35]. Release 3.6.0 reads and writes the file through the reader in this paper [36]. A long exploration writes thousands of such files [37]. A distributed adaptive kinetic Monte Carlo run ships the files between ranks [38]. A public-resource client returns the same files across machines [39].

A surrogate inside the search loop does not replace the file. A Gaussian-process force model is trained on geometries [40]. Pruning that model removes training points and still evaluates the geometry the search is standing on [41]. A Gaussian-process band and a Gaussian-process minimum-mode search both write the geometry they refined [42, 43]. Accelerating the saddle search leaves the checkpoint in place [44]. A Bayesian search places its posterior on stationary points that are geometries [45]. A model container such as metatensor evaluates a geometry; it does not store the search [46]. A universal potential such as PET-MAD consumes a geometry and returns energy and forces [47]. A descriptor library consumes the same geometry and cannot rebuild a mask from the descriptor [48]. Symmetry search can renumber atoms, so an atom id is not recoverable from coordinates alone [49]. A materials-analysis library starts from a stored structure [50]. A materials API returns that structure and does not return the optimiser state that produced it [51].

The file therefore has a job an exchange format does not. A surface calculation holds the slab and lets an adatom move in the plane, so the mask is per atom and per axis [52]. A band and an event table refer to atoms by identity after a file has grouped them by element [53, 54]. Velocities enter a step only with a unit. The natural unit of eOn is $(\mathrm{amu}\cdot\mathrm{\AA}^2/\mathrm{eV})^{1/2}$. On a 10,001-atom slab the committed conversion is 10.1805056712 fs, and the kinetic energy scales by 103.642696 when the same numbers are read as Å/fs [55]. A unit library is how a program names that quantity; the file has to carry the name [56]. Velocity Verlet advances positions with those velocities [57]. A thermostat rescales the kinetic energy it is given [58]. The developers of MDAnalysis stated the documentation problem directly: simulation file formats “are often only poorly documented and just adhere to certain community expectations that can be difficult to understand for outsiders” [59].

Two readers given the same bytes have to return the same state, because

the computation is the data [60]. A second group can rerun that computation only from the file and the program [61]. The file outlives the code that wrote it [62]. A format defined only by one library version collapses when that library changes [63]. The FAIR principles ask for a record another group can reuse [64]. The matching principles for research software ask for a versioned, citable program [65, 66, 67]. Chemical Markup Language and MOSAIC wrote the data model down before the code [68, 69]. checkCIF turned the CIF rules into alerts a later reader can reproduce [70, 71]. CON 3.1 is that treatment of the text eOn already writes.

The contributions are the five requirements, the grammar and corpus that meet them, and one reader that every binding calls. Section 2 measures the requirements on one frame. Section 3 states the grammar. Section 4 is the corpus and the extended XYZ disagreement. Section 5 is the reader and the store. Section 6 resumes the same band through every route. Section 7 reports threads, peers, MPI and the store.

2. Requirements and the failure each one closes

Table 1 states five requirements. Each is a field a later step reads.

Table 1: Requirements on a restart state, the step that reads each field, and the failure measured when the field is missing.

Requirement	Step that reads it	Failure measured here
R1, per-axis mask	Surface search, prefactor	Height constraint dropped; barrier −70.2 meV
R2, persistent id	Band and event table	Id lost on completed foreign round trips
R3, declared time unit	Dynamics	Kinetic energy scaled by 103.642696
R4, binary64 geometry, cell, periodicity	Resume of a search	10^{-9} to 10^{-6} Å, or periodicity rewritten
R5, versioned grammar	Any second reader	One file, four answers from five readers

R1 is a mask for each Cartesian axis of each atom. R2 is an integer stored on the row, so a later frame finds the same atom after the file groups rows by element. R3 is a time unit written in the file. R4 is coordinates, cell and

periodic flags that read back as the values written. R5 is a grammar with a major and a minor, a rule for what a later minor may add, and a corpus that decides disputes.

The short history of an optimiser is outside the five. Limited-memory quasi-Newton updates a Hessian from a few previous steps [72]. The optimisers used on minimum-energy paths keep an analogous history [73]. FIRE carries velocities of the images between steps [74]. A resume can match the coordinates and still take a different next step. Section 6 measures that on one band, and Section 8 states the scope.

2.1. One frame through each format

The frame is a Pt(100) slab with one adatom: the bottom atoms held on every axis, the adatom held in z, ids that are not the row order, and velocities in the natural unit. Section S7 gives the coordinates, the ids and the grading rules. Each row of Table 2 writes that frame with one library and reads it with another. The libraries are ASE 3.29.0 [75, 76], chemfiles 0.10.4 [77], MDAnalysis 2.10.0 [78, 79], and h5py 3.7.0 where the route needs it [80]. The chemfiles concept record covers the project across versions [81]. The 2016 MDAnalysis paper is the toolkit description; the 2.10.0 record is the version in the table.

Table 2: One restart frame through each writer and reader. The frame is the Pt(100) slab with one adatom: the bottom layer is held on every axis, the adatom is held in z, ids are not the row order, and velocities are in the natural unit. R1 to R4 are measured on the round trip. "Kept" means the field came back. "Convention" means it came back only because writer and reader share one library's column name or internal unit, which the format does not define. "Fixed by format" means the format's own definition sets the unit without writing it in the file. R4 gives the largest coordinate change in Å when the coordinates are not bit-identical. Periodicity is the slab's xy-periodic, z-open flags after the read. R5 is the version rule in the format's published definition. "Error" rows failed before any field could be compared; the text gives the failure.

Format	Writer	Reader	R1 mask	R2 id	R3 unit	R4 coordinates	Periodicity	R5 version
CON 3.1	readcon-core	readcon-core	kept	kept	declared	exact	kept	major.minor
CON 3.1	readcon-core	ASE eon reader	error	error	error	error	error	major.minor
CON 3.1, coordinates only	readcon-core	ASE eon reader	full freeze	lost	lost	exact	none	major.minor
legacy CON	ASE eon writer	readcon-core	error	error	error	error	error	none
extended XYZ	ASE	ASE	convention	convention	convention	4.7×10^{-9}	kept	none
extended XYZ	ASE	chemfiles	lost	convention	lost	4.7×10^{-9}	not returned	none
extended XYZ	ASE	MDAnalysis	lost	lost	lost	2.2×10^{-7}	not returned	none
extended XYZ	chemfiles	ASE	lost	lost	lost	4.9×10^{-6}	all periodic	none
ASE trajectory	ASE	ASE	kept	lost	convention	exact	kept	code only
POSCAR	ASE	ASE	changed	lost	fixed by format	4.5×10^{-17}	all periodic	none
H5MD	MDAnalysis	MDAnalysis	lost	lost	declared	2.2×10^{-7}	all periodic	major.minor

The CON 3.1 row keeps every field. Extended XYZ written and read by

ASE returns the mask, the ids and the time unit because both sides share ASE's column names and ASE's internal unit. The extended XYZ grammar does not define those names, so chemfiles drops the mask and MDAnalysis drops the mask and the ids. The ASE writer prints a fixed number of decimals and the coordinates move by up to 4.7×10^{-9} Å. chemfiles writing the same frame moves them by 4.9×10^{-6} Å and returns every axis periodic. MDAnalysis returns single-precision coordinates, a shift of 2.2×10^{-7} Å. The ASE trajectory keeps the mask and the coordinates exactly and drops the id; its time unit is the library's unit, which the file does not name. POSCAR selective dynamics is a per-axis mask in the VASP definition of the ionic restart [82, 83]. The ASE writer turns the held slab into a different mask and writes the adatom free, so this route changes R1. POSCAR has no atom id and no open axis. H5MD written by MDAnalysis declares the velocity unit, writes no mask and no id, stores single-precision coordinates, and marks the open z axis periodic [84]. H5MD sits on HDF5, whose container can hold extra arrays [85]. An extra array carries a mask only when every reader agrees on its name and type, which is the extended XYZ situation again. Section 6 attaches that array and shows which readers return it. The ASE eon reader rejects a CON frame that contains a velocity block. Given coordinates alone it freezes every masked atom on every axis, drops the ids, and returns no periodic axis. The ASE eon writer cannot express a per-axis constraint, so that route produces no file.

2.2. Records that are not this checkpoint

The formats below are in use next to a saddle search. Each one stores a record this checkpoint does not, or omits a field this checkpoint requires. They are the boundary of the problem, and Section 8 keeps that boundary.

TNG is a portable compressed trajectory of a finished molecular-dynamics run [86], and its library and API serve that trajectory [87]. GROMACS writes the production trajectory of a finished run [88]. CHARMM does the same for its own trajectories [89]. AMBER stores trajectories in NetCDF [90, 91]. A portable scientific-data manager stores arrays of a finished dataset [92]. MDTraj analyses those trajectories [93]. VMD displays them [94]. OVITO displays and filters atomistic snapshots [95]. ADIOS2 moves high-performance output during a run [96]. Essential-dynamics compression keeps principal components and discards the exact coordinates a resume needs [97]. d-SEAMS analyses a finished trajectory for structure [98]. An MPI-parallel

H5MD analysis reads that finished trajectory; it is a different job from a checkpoint read [99]. Zarr specifies chunked arrays for analysis [100].

The Protein Data Bank archives a curated structure [101]. The worldwide PDB is the organisation of that archive [102]. mmCIF is the ontology of the same archive [103]. A LAMMPS dump is a snapshot whose units live in the input deck [104]. Atomsk and Open Babel convert columns that both dialects define [105, 106]. cclib and IOData parse quantum-chemistry outputs into documented objects [107, 108]. TREXIO stores a wave function [109]. QCArchive stores quantum-chemistry results [110]. OPTIMADE, NOMAD, Catalysis-Hub, Transition1x and OC20 distribute structures or reaction endpoints after the search [111, 112, 113, 114, 115]. PACKMOL builds an initial box and does not record a search [116]. PLUMED accumulates collective variables on a running trajectory [117]. ESPResSo writes the checkpoint of its own integrator [118]. i-PI, in its original interface and in the 2.0 and 3.0 force engines, exchanges positions during a running step [119, 120, 121]. The MolSSI driver interface does that exchange on the fly [122]. The stopped job still resumes from a file. The electronic-structure community has argued for one library per concern with a stable interface, which is the split used here between the grammar and the store [123].

3. The CON 3.1 specification

A CON file is a sequence of text frames. Listing 1 is one nine-atom frame. The survey frame of Table 2 uses the same columns; its ids and its coordinate offset are in Section S7.

Listing 1: A nine-atom Pt frame in CON 3.1. Line 2 declares the version, the units and the periodicity. The velocity block is abbreviated.

```
Pt(100) 2x2x2 with one Pt adatom
{"con_spec_minor":1,"con_spec_version":3,"pbc":[true,true,false],"sections
    ":["velocities"],"units":{"energy":"eV","length":"angstrom","mass":"
    amu","time":"(amu*angstrom^2/eV)^0.5"}}
5.5437171645025325e0 5.5437171645025325e0 1.396e1
9e1 9e1 9e1


1
9
1.95084e2
Pt
Coordinates of Component 1
```

```
1.3859292911256331e0 1.3859292911256331e0 6e0 7 0
4.1577878733769e0 1.3859292911256331e0 6e0 7 1
1.3859292911256331e0 4.1577878733769e0 6e0 7 2
4.1577878733769e0 4.1577878733769e0 6e0 7 3
0e0 0e0 7.96e0 0 4
2.7718585822512662e0 0e0 7.96e0 0 5
0e0 2.7718585822512662e0 7.96e0 0 6
2.7718585822512662e0 2.7718585822512662e0 7.96e0 0 7
1.3859292911256331e0 1.3859292911256331e0 9.76e0 4 8

Pt
Velocities of Component 1
1.4685859972256804e-3 6.275175116609037e-4 -1.8115929554652053e-3 7 0
3.748769356553213e-3 2.43327727576049e-3 -1.3238979663774354e-3 4 8
```

Line 1 is free text. Line 2 is one JSON object [124]. A bulk JSON parser reaches gigabytes per second on arrays of objects, so this single header line is not the cost measured in Section 7.1 [125]. Lines 3 and 4 are the cell lengths and the angles in degrees. The type count, the per-type counts and the masses follow. Each coordinate row is x, y, z, the mask and the id. Lengths and angles rebuild the box with a along x and b in the xy plane,

$$\mathbf{a} = a\,\hat{\mathbf{x}}, \qquad \mathbf{b} = b\cos\gamma\,\hat{\mathbf{x}} + b\sin\gamma\,\hat{\mathbf{y}}, \tag{1}$$

after which $\mathbf{c}$ is fixed by α and β so the cell is right-handed [126]. When line 2 carries lattice vectors, those rows are the cell. A missing periodicity flag means periodic on every axis.

R1, the mask.. Column 4 is the integer of Table 3. Mask 0 is free, mask 4 fixes z only, and mask 7 fixes every axis. A float spelling, a sign, or a value outside 0 to 7 is an illegal flag. Under version 1 the column is 0 or 1, so masks 4 and 7 cannot be stored as written.

Table 3: Mask bits from version 2. Version 1 stores 0 (free) or 1 (fixed on x, y and z) in the same column.

Bit	Axis	Value
0	x	1
1	y	2
2	z	4

R2, the identity.. Column 5 is an integer from 0 through $2^{64} - 1$. Writers emit ids unique within a frame. A missing id column takes the zero-based row index.

R3, the time unit.. From minor 1, a velocity block declares `units.time`. The natural unit is spelled `(amu*angstrom^2/eV)^0.5`. Versions 1, 2 and 3.0 are read in the natural unit. An fs label on those older files is not trusted: producers of that era wrote natural-unit numbers under an fs label [36].

R4, the numbers.. Readers convert each finite decimal to binary64 with correct rounding [127, 128, 129, 130]. The default writer prints the shortest decimal that parses back to the same value. That printer is the end of the line that runs from the early correct algorithms through Grisu, Errol and Ryū [131, 132, 133, 134, 135, 136]. Six fixed decimals are an optional writer mode. gzip and zstd wrap the same bytes, and a reader recognises the wrapper by content [137, 138]. The wrapper is part of the grammar: a conforming reader accepts the compressed file and returns the same frames.

R5, the version.. Table 4 lists the reserved keys of the JSON line. A reader accepts a later minor of a known major, keeps an unknown key on rewrite, may skip an unknown section whose width is declared, and rejects an unknown major. H5MD uses the same split for its version pair [84]. Table 5 is the history of the grammar. Table 6 is the sections reserved at minor 1. Table S1 is the reader rules. The grammar of one frame is the 50-rule listing in Section S13. The prose specification wins where the listing and the prose disagree.

3.1. What the text costs

eOn has written CON since its first releases, so the campaigns already on disk are in this grammar [34]. The server and its clients exchange those files across machines, and the bytes do not depend on endianness [38]. A diff of a text checkpoint shows a changed mask or id. The JSON line takes a new key without a new grammar, and a reader keeps a key it does not know [124].

Table 7 is the byte cost on the committed size table: one 218-atom frame, and the same frame repeated 10,000 times, raw and after gzip at level 6. The repeated file is the same text, so its gzip column is smaller than the gzip column of a trajectory whose frames differ. That is the measurement in the table. One CON frame is 41.99 bytes per atom, against 83.64 for extended XYZ written by ASE with the mask column, 314.39 for one H5MD frame written by MDAnalysis, and 95.40 for one H5MD frame written by h5py with

Table 4: Reserved metadata keys of the JSON header line.

Key	Type	Key	Type
`con_spec_version`	int	`neb_bead`	int
`con_spec_minor`	int	`neb_band`	int
`section_arity`	object	`convergence_fmax`	number
`sections`	array	`convergence_energy`	number
`validate`	bool	`converged`	bool
`generator`	string	`fmax`	number
`units`	object	`bonds`	array
`pbc`	array	`storage_dtypes`	object
`lattice_vectors`	array	`chemfiles_atom_names`	array
`energy`	number	`chemfiles_atom_types`	array
`potential`	object	`chemfiles_atom_properties`	array or object
`frame_index`	int	`time`	number
`timestep`	number		

Table 5: CON versions. The date is the day the rule was fixed.

Version	Date	Change
1	eOn text	Column 5 present. Column 4 is 0 or 1. No JSON line.
2	2026-03-25	A JSON header line, ids, bit masks, declared sections.
3.0	2026-06-27	Length and energy units required. No minor.
3.1	2026-10-11	Minor 1. A velocity block declares the time unit.

binary64 positions, a mask dataset and an id dataset. At 10,000 copies the raw CON cost stays 41.99 bytes per atom. H5MD amortises its container and falls to 12.61 and 24.04 bytes per atom. gzip of the repeated CON file reaches 0.39 bytes per atom. H5MD is the record for a long trajectory, where binary storage and chunked reads pay for the container [84]. CON is the record for the checkpoint, where one frame has to keep the mask, the id and the unit.

4. Conformance and drift

The corpus is the executable form of the grammar, on the model of checkCIF [71, 70]. The committed manifest summary is 10 legal files and 8 illegal files. The files on disk split as in Table 8: two header-and-unit frames,

Table 6: Reserved sections at minor 1. Each data row carries its values, then the mask and the id.

Section	Values	Rule
Velocities	$v_x\, v_y\, v_z$	Time unit from minor 1
Forces	$f_x\, f_y\, f_z$	
Energies	one value	Name `energies`
Charges	one value	
Spins	one value	
Magmoms	$m_x\, m_y\, m_z$	
Displacements	$d_x\, d_y\, d_z$	`units.length`
Spreads	$s_x\, s_y\, s_z$	`units.length`, each at least 0

Table 7: Bytes per atom of the 218-atom frame, alone and repeated 10,000 times, raw and after gzip at level 6. CON 3.1 is the default readcon-core writer, extended XYZ the ASE writer with the mask column, H5MD (MDAnalysis) its writer in single precision, and H5MD (h5py) binary64 positions with mask and id datasets. The repeated file compresses better than any real trajectory would.

	One frame		10,000 frames	
Format	raw	gzip	raw	gzip
CON 3.1	41.99	6.07	41.99	0.39
extended XYZ (ASE)	83.64	6.65	83.64	1.13
H5MD (MDAnalysis)	314.39	9.04	12.61	0.12
H5MD (h5py)	95.40	9.66	24.04	0.16

five mask frames and three empty frames, plus eight illegal files. A conforming reader accepts the first group and rejects the second. The same summary names five bindings, one per language of the C interface. The measured test of those languages on a resume is the 25-pair matrix in Section 6.

Table 9 applies R3 and R5 to seven headers of one velocity frame. Python and C follow the rule on each row. They reject a minor-1 velocity block with no time unit, an unknown major, a missing version and a mask of 9, and they keep an unknown key from a later minor. The ASE eon reader has no version rule. Given the coordinate block, it accepts all seven, including the illegal mask and the unknown major.

Extended XYZ has a published layout and parsers in several languages, so it is the text format to compare on R5. Table 10 gives five readers seven three-atom files. The readers are ASE 3.29.0, libAtoms 0.4.6 through its C path and

Table 8: Conformance corpus. Counts are the committed manifest summary.

Category	Files	What a reader must do
Header and units	2	Accept a version-2 header and a version-3 units line.
Column-4 masks	5	Accept version-1 full freeze and version-2 per-axis flags.
Empty frame	3	Accept a type count of zero.
Illegal	8	Reject the file.

Table 9: Header variants of one velocity frame. Rule is what the specification requires. The ASE eon reader is given the coordinate block only, because it stops at a velocity section.

Variant	Rule	Python	C	ASE eon
3.1, natural unit declared	accept	accept	accept	accept
3.1, no time unit	reject	reject	reject	accept
3.0, labelled fs	accept (natural)	accept	accept	accept
3.2, unknown key	accept, keep	accept, kept	accept	accept
4.0	reject	reject	reject	accept
missing version	reject	reject	reject	accept
3.1, mask 9	reject	reject	reject	accept

its Python path, chemfiles 0.10.4, and MDAnalysis 2.10.0. On the reference frame the five readers return four outcomes: ASE and the libAtoms C path accept it, the libAtoms Python path changes the mask booleans, chemfiles moves one coordinate by one unit in the last place and returns no mask, and MDAnalysis returns single precision with no mask and no ids. The correctly rounded binary64 value of the coordinate 2.7718585822512662 has bit pattern `40062cc4313f3fb9`; chemfiles returns `40062cc4313f3fb8` (Section S7). A Fortran exponent `6.0d0` is read by the libAtoms C path, returned as NaN by the libAtoms Python path, and rejected by the other three. A plain comment line is non-periodic to ASE, periodic to the libAtoms C path, and rejected by the libAtoms Python path. The readers follow their own documentation. The grammar does not decide these cases, and the formats share no corpus that would.

Table 10: Five extended XYZ readers on seven three-atom files. Each file departs from the reference frame in one respect. ✓: positions correctly rounded, periodicity as written, mask and ids returned as written. E: the read failed. u: a coordinate one unit in the last place from the correctly rounded value. f: single-precision coordinates. n: a coordinate returned as NaN. p: periodicity differs from the file, or from the grammar's default when the key is absent. b: the mask came back with different booleans. m, i: the reader returns no mask, or no ids.

File	ASE	libAtoms C	libAtoms Py	chemfiles	MDAnalysis
reference frame	✓	✓	b	um	fmi
no pbc key	✓	✓	b	um	fmi
plain comment line	✓	p	E	u	f
Fortran exponent `6.0d0`	E	✓	nb	E	E
`True`/`False`	✓	✓	b	um	fmi
quoted value with `=`	✓	✓	b	um	fmi
`units` key	✓	✓	b	um	fmi

5. Reference implementation

readcon-core 0.17.0 is the reference reader and writer [139]. The concept record covers every version [140]. The store measurements in Section 7.4 use 0.16.2 [141]. Figure 1 is the layering. One Rust core parses and writes. One C interface is the waist every other caller uses. C++ wraps it for eOn. Python exposes array views [142]. Julia calls the C symbols through its foreign-function interface [143]. Fortran 2008 reaches the same symbols through C interoperability. chemfiles took the same route for trajectories, one core and several language bindings [77]. Rust gives the core memory safety without a garbage collector on the far side of a C interface [144, 145]. Decimal fields go through a correctly rounded conversion on that shared path [146, 147]. A second parser would be a second implementation. The public Fortran wrapper shows that a binding can still disagree with the core; Section 6 records the status it returns.

The caller owns a frame handle and frees it. Structure-of-arrays accessors return a pointer into that frame. Absent numeric metadata is a NaN sentinel and is not written back. The reader records the byte offset of each frame. The scan that finds those offsets is serial, because each start depends on the counts before it. Each frame is then parsed on its own, and the frames return in file order. The Python binding releases the interpreter lock during the parse and takes it back to build objects.

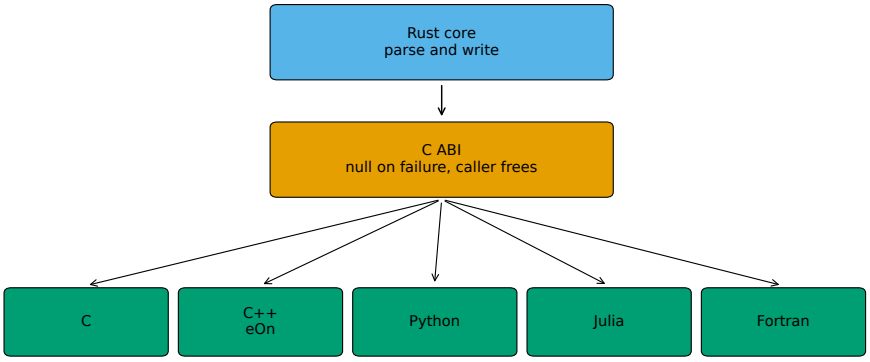


Figure 1: The Rust core parses and writes CON. The C interface is the waist. eOn 3.6.0 calls the C++ wrapper.

Listing 2: A round trip in Python. The default printer preserves binary64. Six decimals are a separate argument.

```
frames = readcon.read_con("slab.con")
readcon.write_con("rt.con", frames)
```

readcon-db is the campaign store for that text [148]. It writes each CON frame into LMDB, posts an xxHash3-128 of the text, and keeps secondary indexes on atom count, composition, energy, maximum force and section presence. A select touches the index and then the frames the index names. Identical texts share one hash entry, and each frame keeps its own stored text, so the stored blob bytes equal the raw file. gzip and zstd remain properties of the CON bytes, recognised by content before ingest. The MPI paths are a single writer, which inserts in global frame order, and a sharded ingest. Section 7.3 measures both. Two behaviours of the bindings are part of the same test, and both are visible in the evaluation record. Python `ConCorpus.select` always applies an atom-count range, including the default range from 0 to $2^{32} - 1$, so a Python query walks the atom-count index. The timed queries use the Rust select and do not add that predicate. `readcon.iter_con` skips a leading newline, so an empty user line shifts the header and the parse fails. `read_con` on one frame keeps that empty line. The specification says an empty first header line is the user line and is kept. The evaluation writes the user line `cpc-eval` so the iterator can read the corpus. `from_ase` leaves the user line empty, which is legal CON.

6. Case studies

Every route below uses one protocol. An EMT Pt(100) slab, three layers of 3×3, has its bottom layer held on every axis and one adatom held in z. The adatom hops between neighbouring hollows. Endpoints were relaxed to $0.002\,\mathrm{eV/Å}$. A climbing-image band of five movable images uses the improved tangent [5, 6] and fixed-step steepest descent on the projected force. Steepest descent stores no step history, so a difference after the resume comes from the file. The band is interrupted after 25 steps, written, read back, and resumed to $0.01\,\mathrm{eV/Å}$. Uninterrupted, it converges in 60 steps with barrier $0.644561366\,\mathrm{eV}$ and 56 free modes on the middle image. Table 11 is every completed route. Figure 2 is the three bands whose geometries are in the committed figure. The 25 language pairs are Table S4.

Table 11: Climbing-image band of the EMT Pt(100) hop, stopped after 25 steepest-descent steps, written and read back, then resumed to $0.01\,\mathrm{eV/Å}$. File is the largest coordinate change on that round trip. Modes are the free Cartesian degrees of freedom of the middle image (56 when the masks survive). $\Delta E^{\ddagger}$ and the displacement are against the uninterrupted band, whose barrier is $0.644561366\,\mathrm{eV}$. [a]Not converged at 4000 steps. The 25 language pairs are Table S4.

Route	File (Å)	Modes	Steps	$\Delta E^{\ddagger}$ (eV)	Max displacement (Å)
uninterrupted	–	56	60	0	0
CON 3.1, shortest round trip	0	56	60	-5.68×10^{-14}	2.66×10^{-15}
CON 3.1, six decimals	4.90×10^{-7}	56	60	-4.14×10^{-7}	6.22×10^{-7}
CON 3.1 read by the ASE eon reader	0	54	4000[a]	-3.52×10^{-1}	5.43×10^{0}
same, periodicity restored by hand	0	54	127	-2.93×10^{-6}	4.21×10^{-2}
extended XYZ (ASE)	5.00×10^{-9}	56	60	1.09×10^{-9}	4.94×10^{-9}
ASE trajectory	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
POSCAR (ASE)	4.90×10^{-17}	57	98	-7.02×10^{-2}	1.60×10^{-1}
legacy CON (ASE eon writer)	no file: TypeError, FixCartesian is not a per-axis constraint				
ASE trajectory, id array	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
H5MD MDAnalysis to MDAnalysis	4.60×10^{-7}	84	97	-7.07×10^{-2}	1.43×10^{-1}
H5MD MDAnalysis to h5py	4.60×10^{-7}	56	60	-1.98×10^{-7}	3.67×10^{-7}
H5MD h5py to h5py	0	56	60	-2.31×10^{-14}	3.55×10^{-15}
H5MD h5py to MDAnalysis	4.60×10^{-7}	84	97	-7.07×10^{-2}	1.43×10^{-1}

The shortest CON round trip changes no coordinate. The barrier moves by $-5.68 \times 10^{-14}\,\mathrm{eV}$ and the images by at most $2.66 \times 10^{-15}\,\mathrm{Å}$. Each of the 25 language pairs also changes no coordinate, keeps 56 modes, and finishes in 60 steps at the same printed barrier. Their barrier shift is $-2.31 \times 10^{-14}\,\mathrm{eV}$ and their largest displacement is $3.55 \times 10^{-15}\,\mathrm{Å}$. The two floors are both at the level of rebuilding the calculator. They are different numbers, and the table keeps them separate. Six decimals move the file by $4.90 \times 10^{-7}\,\mathrm{Å}$, the

barrier by -4.14×10^{-7} eV, and the images by 6.22×10^{-7} Å, and the band still converges in 60 steps with 56 modes. Extended XYZ moves the file by 5.00×10^{-9} Å and the barrier by $+1.09 \times 10^{-9}$ eV, with the same step count and the same mode count. The ASE trajectory matches the language-pair floor. An extra id array on that trajectory does not survive the round trip, and the mode count stays 56 because the mask does.

POSCAR moves the file by 4.90×10^{-17} Å and drops the height constraint, so the middle image has 57 free modes. The band converges in 98 steps, 0.160 Å away, with a barrier 7.02×10^{-2} eV lower. That is the 70.2 meV in the abstract. The legacy ASE eon writer raises `TypeError` because `FixCartesian` is not a per-axis constraint in that writer, and there is no file to resume. The ASE eon reader keeps 54 modes, runs to the 4000-step cap without converging, and finishes 0.352 eV and 5.43 Å away. Restoring periodicity by hand leaves the mask collapse in place: 54 modes, 127 steps, a barrier shift of -2.93×10^{-6} eV, and a displacement of 4.21×10^{-2} Å.

The H5MD block in the table is a second measurement, distinct from the survey row. The writer attaches `/particles/trajectory/fixed` as a Boolean $N \times 3$ array, an `id` array, the velocity unit Å ps^{-1}, and the boundary `[periodic, periodic, none]`. MDAnalysis reading that file, whether it wrote the file or h5py did, returns 84 free modes and a barrier 7.07×10^{-2} eV lower, with a file shift of 4.60×10^{-7} Å. The mode count is the loss of the mask and the id. h5py reading the MDAnalysis file keeps 56 modes, finishes in 60 steps, and moves the barrier by -1.98×10^{-7} eV. h5py reading its own file is bit-exact and lands on the language-pair floor. The coordinate gap between the MDAnalysis file and the h5py file is the single-precision write.

The Fortran rows in Table S4 call `rkr_frame_add_atom_full` in the C header order $(x, y, z, f_x, f_y, f_z, \mathrm{id}, \mathrm{mass})$, with velocity and force. Those rows pass. The public Fortran `add_atom` binds `rkr_frame_add_atom_with_fixed_mask` as $(x, y, z, \mathrm{id}, \mathrm{mass}, f_x, f_y, f_z)$. The C function takes $(x, y, z, f_x, f_y, f_z, \mathrm{id}, \mathrm{mass})$. A write through the public wrapper returns status -4, with atom id 1 repeated, and the wrapper has no velocity argument. The in-repository Fortran module was left unchanged. The passing Fortran rows are the full C entry point. The public wrapper is a failed write, measured the same way as the legacy ASE writer that produced no file.

6.1. The mask and the prefactor

The symmetric hop can sit on the straight line, so a lost lateral constraint need not move the barrier by much. A second band, seven images on the

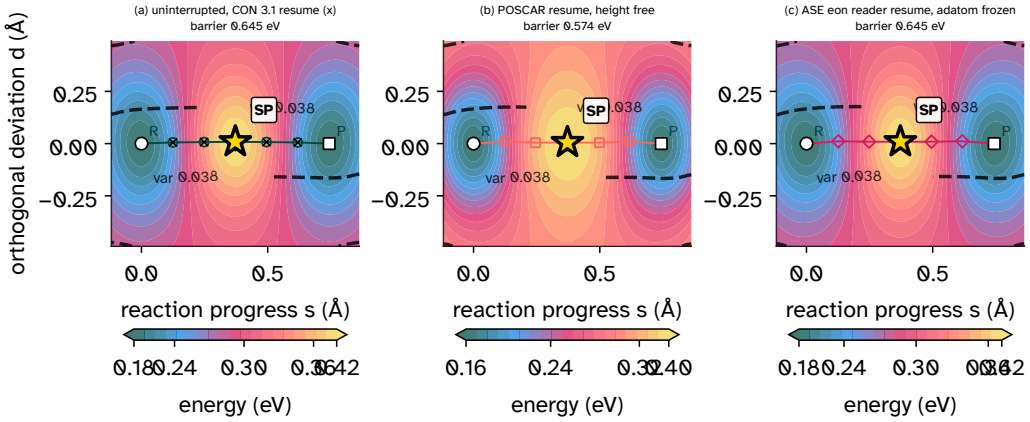


Figure 2: Resumed climbing-image bands of the EMT Pt(100) hop on the (s, d) plane [149]. Each panel is the energy surface of its own band. (a) Uninterrupted, with the CON 3.1 resume as crosses. (b) POSCAR, height constraint dropped. (c) ASE eon reader with periodicity restored by hand. The projection puts the saddles near one another. The routes differ in barrier and in free modes, as Table 11 records.

smaller slab, was converged with FIRE to 0.01 eV/Å twice [74]. ASE 3.29.0 has no eOn Morse calculator here, so the relaxation, the band and the Hessian use EMT. With the height held, the barrier is 0.36659812266626624 eV on a path of 2.9004016648060382 Å. The middle image has 56 free modes and one imaginary mode, −4.076276454192005 meV, with 0.9872893953755115 of its weight on the adatom's lateral motion. The Vineyard prefactor of that Hessian is $3.2438730218892426 \times 10^{10}$ Hz [2]. Kellogg's measured Pt(001) self-diffusion rate is a different quantity, an experimental rate on this face [150]. FIRE took 26 steps and then 16 climbing steps.

The ASE eon reader turns the height-held column into `FixAtoms`, so the adatom is frozen on every axis. The default interpolation then raises `RuntimeError` because the constraints change the path, and the band was built with the constraints ignored during interpolation. FIRE took 26 steps and then 14 climbing steps, reached 0.007095 eV/Å, and left the adatom on the straight interpolation. The relative peak is 0.3665836803750313 eV on a path of 2.9595547559012947 Å. The middle image has 54 free modes and no imaginary mode, so the Vineyard ratio is undefined. Figure 3 is the two bands. The barriers differ in the fifth digit. The prefactor exists for one reading and does not exist for the other.

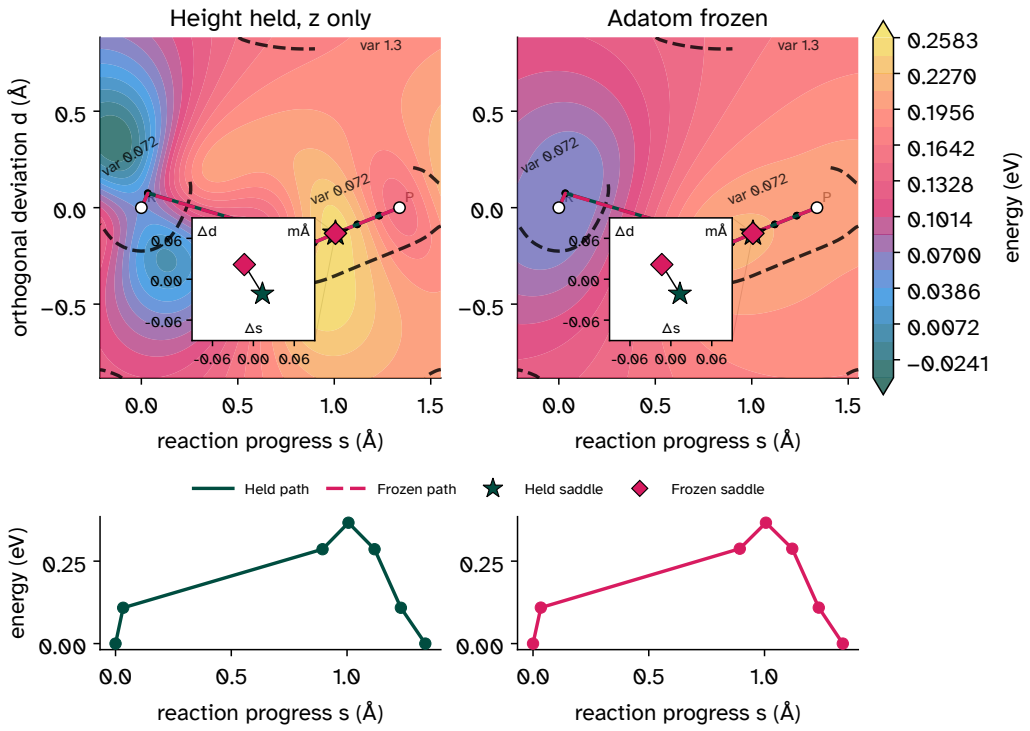


Figure 3: EMT climbing-image bands for a Pt adatom hop on Pt(100), each mask on its own (s, d) map [149]. Height held: 0.366598 eV, 56 free modes, one imaginary mode. ASE eon reading: 0.366584 eV, 54 free modes, no imaginary mode.

6.2. eOn resuming a Morse band

An eOn 3.6.0 client ran a Morse Pt potential on a 28-atom slab, bottom atoms at mask 7, five movable images, steepest descent [36]. The band section selects the optimiser. This run sets steepest descent there. When the total number of optimiser steps matches, the shortest round trip reproduces the uninterrupted barrier, 0.63493 eV, with barrier difference 0 and a largest minimum-image displacement of $8.881784197001252 \times 10^{-15}$ Å. Six decimals give barrier 0.634931 eV and a largest minimum-image displacement of $4.918050526825368 \times 10^{-7}$ Å. Figure 4 is that comparison. Section S1 gives the per-image displacement.

A control file left the default band optimiser in place. Its recorded optimiser field is `sd`. The file note names the default as L-BFGS and says the iteration tables were not stored [72, 73]. Both printers of that control finish at 0.670723 eV. The uninterrupted barrier in the same file is 0.63361 eV, so

the difference is 0.037113 eV. The two printers agree with each other. They do not agree with the uninterrupted barrier. The checkpoint does not carry the history that would make the next step the uninterrupted step.

A dimer started from the round-trip geometry stops for the same reason as the uninterrupted dimer, too many iterations. Both report saddle energy -112.618941 eV and curvature -10.613681. Six decimals give -112.618944 eV and -10.613677, an energy shift of -3×10^{-6} eV. The comparison is of the stopped search.

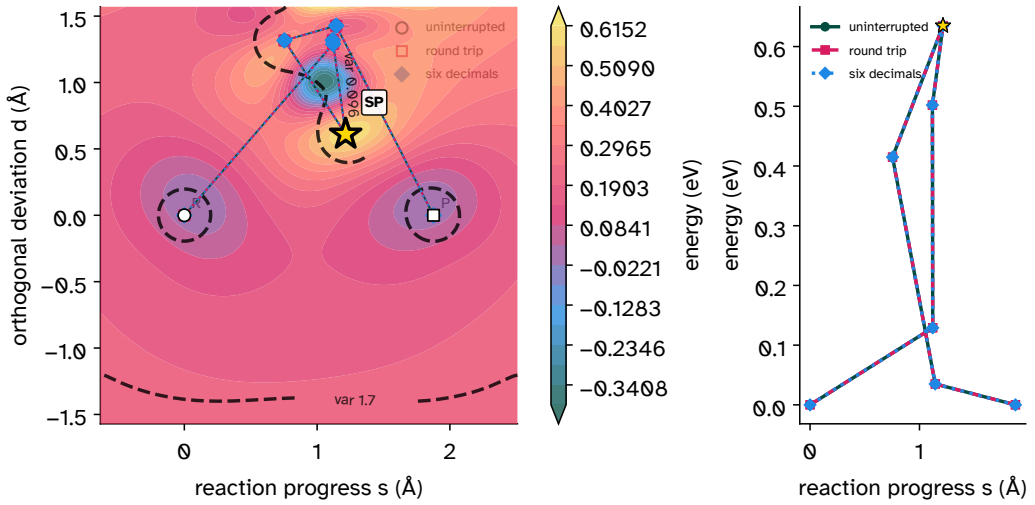


Figure 4: Final images of the Morse band on the (s, d) plane [149]. With the step count matched, the shortest round trip returns the uninterrupted barrier, 0.63493 eV.

6.3. The velocity unit and identity

On the 28-atom slab the kinetic energy is 0.8878595057627452 eV in the natural unit and 92.0201533142073 eV when those numbers are read as Å/fs. The ratio is 103.64269652680504. One velocity Verlet step of 1 fs then moves the atoms by an RMSD of 0.09556596694568728 Å, maximum 0.31544697368347396 Å [57]. The same factor appears on a separate 10,001-atom slab at 300 K: the natural unit is 10.1805056712 fs, the kinetic energy is 390.085819 eV, and the misread value is 40429.545793 eV, a factor 103.642696. Section S4 plots the step.

The public HCN and HNC endpoints carry the unit and the identity in one file [151, 152]. readcon-core on frame 2 returns masks `000 000 100`, ids 2

`0 1`, and the declared natural unit. The ASE eon reader stops on the velocity block with `ValueError` at the word `Velocities`. Given coordinates only, it returns the three atoms as free, free, and fixed on every axis, and it returns no ids.

7. Performance

Timings below are medians. Where a bias-corrected and accelerated interval is reported, it uses 9999 resamples [153]. The host, the compiler and the thread pin are part of the result, because a changed stack changes the number [154, 155]. Parallel runs are compared at fixed work or fixed work per rank, with the binding named [156]. Several peer ratios are tested together, and the p values are Holm-adjusted [157]. Every row of the thirty-pair table has Holm $p = 0.0016$.

7.1. Threads

The file is 10,000 copies of one 218-atom frame, 167,710,000 bytes, on an Intel Xeon, family 6, model 207, with 8 hardware threads. The timed binary was built with rustc 1.88.0. OpenMP and BLAS are pinned to one thread outside the pool under test. Table 12 is the warm-cache read. The native Rust pool finishes in 0.125501 s, 1336 MB/s, 79681 frames/s, which is 12.6 μs per frame. The C++ wrapper eOn calls finishes in 0.13315 s. The Rust-to-C++ ratio of those medians is 0.904, with interval $[0.815, 0.993]$. At one thread the two medians are 0.483641 s and 0.484879 s, ratio 0.991. Building a Python object per frame takes 1.234571 s. An estimate of the object construction alone is 0.754 s of that. One EMT force call on this host has median 5.2623×10^{-5} s, so the native read of the whole file is about 2385 such calls, 0.24 calls per frame. The in-process Morse timing from the band study is 3.036×10^{-4} s per call, and the same file is about 413 of those calls. i-PI's force engine is the setting in which that comparison matters: the client pays a force call on every step and a file read when the job stops [121].

A separate five-repeat sweep on the same file sets the pool size. Rust goes from 0.50026 s at one thread to 0.11503 s at eight, a speedup of 4.35. C++ goes from 0.50177 s to 0.12793 s, a speedup of 3.92. Python full objects go from 1.2755 s to 1.1070 s, a speedup of 1.15. The file read itself, measured by sending the bytes to the null device, has median 0.018511 s, 9059.8 MB/s. Reading the bytes in the process takes 0.094482 s, and scanning frame boundaries takes 0.034420 s. Those two stages are 0.267 of the one-thread native median.

Table 12: Warm read of 10,000 frames, 167,710,000 bytes. Native rows are 15 repeats. The Python row is 9 repeats. The interval is a bias-corrected and accelerated 95 percent interval of the median.

Reader	Median (s)	MB/s	Interval (s)
Rust, automatic pool	0.125501	1336	[0.115354, 0.137084]
C++ wrapper, automatic pool	0.13315	1260	[0.120279, 0.153288]
Rust, 1 thread	0.483641	346.8	[0.473807, 0.486509]
C++ wrapper, 1 thread	0.484879	345.9	[0.480843, 0.537422]
Python, full objects	1.234571	135.8	[1.216353, 1.303821]

Figure 5 is the automatic-pool rates. Figure 6 places the scaling-block rates under that read rate. The serial fraction drawn there is (file read + boundary scan)/(one-thread native read).

A second Python sweep, thirty interleaved pairs on the same 10,000-frame file, is a different campaign from the five-repeat block. Its one-thread median is 1.35258 s. Two threads give speedup 1.148 with interval $[1.126, 1.157]$, four give 1.209 with $[1.194, 1.223]$, and eight give 1.284 with $[1.263, 1.313]$. Hashes of the returned fields at 1, 2, 3, 4 and 8 threads are identical. The 1.15 and the 1.28 are different experiments, and the text keeps both.

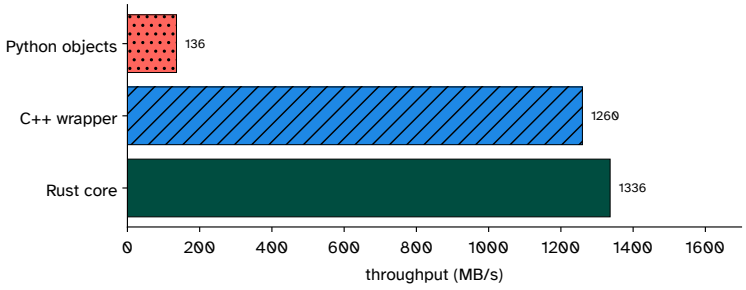


Figure 5: Warm-cache rate for the automatic Rust pool, the C++ wrapper and the Python full-object read, from the medians in Table 12.

7.2. Peers

Table 13 is readcon-core through its Python binding at one worker, divided by the peer, so a ratio below 1 means readcon-core finished sooner. Campaign A, rustc 1.88, thirty interleaved pairs, compares the ASE eon reader, chemfiles XYZ, h5py and MDAnalysis H5MD. On 10,000 frames the ratios are 0.337

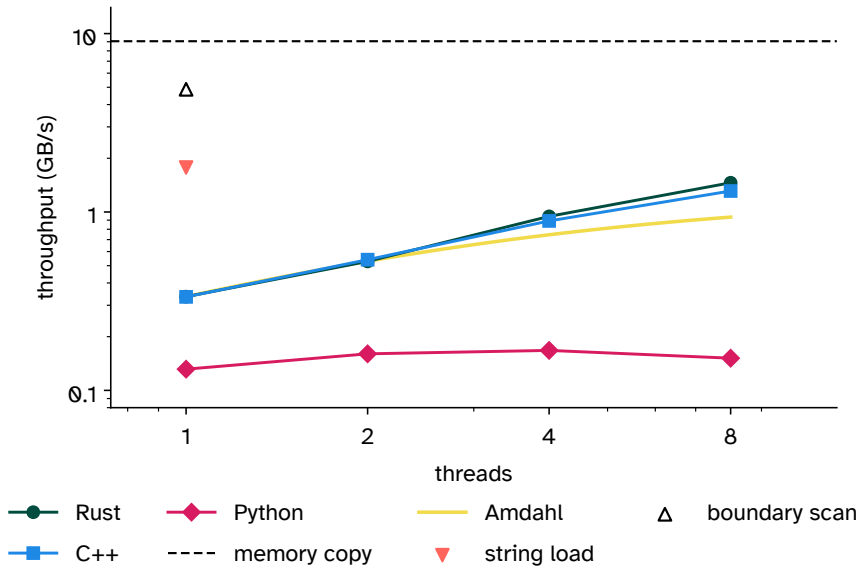


Figure 6: Scaling-block rates for Rust, C++ and Python. The dashed line is the hot-cache copy to the null device, 9.06 GB/s. The curve is Amdahl's law anchored at the one-thread Rust rate of the scaling block. Its serial fraction is 0.267, the in-process file read plus the boundary scan, divided by the one-thread native median. The filled triangle is the string-load rate and the open triangle is the boundary-scan rate.

against ASE eon (readcon-core 1.269 s, ASE 3.745 s), 2.15 against chemfiles XYZ, 67.1 against h5py (peer 0.01899 s), and 0.207 against MDAnalysis H5MD (peer 6.179 s). h5py returns one numeric array. The CON reader returns the mask, the id, the cell and the periodic flags. The ratio is the cost of that parse against a raw array read. Campaign B, rustc 1.99, adds ASE extended XYZ and the ASE trajectory on a second host pass. The ASE eon ratio on the same 10,000-frame file is 0.348, against 0.337 in campaign A. Absolute readcon-core times differ between the campaigns, near 2.0 s in B and 1.27 s in A, so only the reproduced ratio is compared. XYZ and H5MD peers receive coordinates of the same frames, written before the timed loop. Section S9 extends the table to the 1-frame and 100-frame files. On one frame the open dominates, and h5py and chemfiles lose the lead they have on the long file.

Table 13: Paired ratios of the one-worker Python read to each peer, readcon-core time over peer time, so a value below 1 means readcon-core finished sooner. Each cell is 30 interleaved pairs; the interval is a bias-corrected and accelerated 95 percent interval of the median. The fields column says what the peer returns. The two ASE rows beyond the eon reader come from a second campaign on a shared host, in which the ASE eon ratio on the 10000-frame file reproduced at 0.348 against 0.337.

Payload	Peer	Fields	Ratio	Interval
10000 frames	ASE eon	same CON text	0.337	[0.335, 0.345]
10000 frames	chemfiles XYZ	coordinates	2.15	[2.14, 2.17]
10000 frames	h5py	one binary array	67.1	[66.5, 67.3]
10000 frames	MDAnalysis H5MD	coordinates	0.207	[0.204, 0.213]
Pt slab	ASE eon	same CON text	0.233	[0.231, 0.241]
Pt slab	chemfiles XYZ	coordinates	1.14	[1.13, 1.16]
Pt slab	h5py	one binary array	6.05	[5.91, 6.27]
Pt slab	MDAnalysis H5MD	coordinates	0.394	[0.387, 0.404]
10000 frames	ASE extended XYZ	ASE atoms	0.392	[0.338, 0.42]
10000 frames	ASE trajectory	ASE atoms	1.02	[0.948, 1.08]
Pt slab	ASE extended XYZ	ASE atoms	0.184	[0.18, 0.192]
Pt slab	ASE trajectory	ASE atoms	3.19	[3.08, 3.28]

7.3. MPI

The MPI campaign ran on the same Xeon family, 8 cores, about 47 GB, a virtio block device with ext4. Ranks 16 and 32 were not run. MPICH 4.2.3 used the ch4:ofi device, `mpiexec -bind-to core`, with UCX and libfabric on TCP. `RAYON_NUM_THREADS` and `OMP_NUM_THREADS` were 1, so a rank does not spawn a second pool. rustc was 1.99.0. readcon-core was 0.17.0 at `af8aabb`. readcon-db was 0.1.8 at `55214d1`. Each cell is five repeats. Cold repeats drop the page cache. Warm repeats follow an untimed warmup. The median is the third of the five sorted times. Every one of the 480 repeats matched the serial bytes. The tables report the harness time of the distributed section.

Strong scaling holds the file fixed. The CON file is 1,583,691 copies of a small frame, 3,167,382 frames, 1,073,742,498 bytes. The database file is 30,965 frames of 1,000 atoms, 1,074,071,389 bytes, ingested four times (4,296,284,756 bytes) with batch 4096. The sharded path uses four shards. Table 14 is the warm medians. Shared read goes from 9.088 s to 1.504 s, a ratio 6.04. Scatter goes from 9.223 s to 1.836 s, ratio 5.02. MPI-IO write goes from 8.369 s to 1.416 s, ratio 5.91. Gather write goes from 9.490 s to 2.152 s, ratio 4.41. One-writer ingest goes from 31.699 s to 16.810 s, ratio 1.89. Four-shard ingest goes from 32.549 s to 18.805 s, ratio 1.73. Cold shared read goes from 9.904 s

to 2.113 s. Figure 7 is the warm speedup. The full rank list, cold and warm, is in Section S12.

Table 14: Strong scaling, warm page cache. Each cell is the median of five repeats. t_1/t_8 is the one-rank median divided by the eight-rank median. The file is 1,073,742,498 bytes. readcon-core holds 3,167,382 frames. readcon-db ingests four copies of a 1,074,071,389-byte trajectory (30,965 frames of 1,000 atoms), batch 4096. One thread per rank. Every output matches the serial bytes.

Path	1 rank (s)	8 ranks (s)	min–max at 8	t_1/t_8
read, shared file	9.088	1.504	1.416–1.570	6.04
read, scatter	9.223	1.836	1.810–1.875	5.02
write, MPI-IO	8.369	1.416	1.366–1.491	5.91
write, gather	9.490	2.152	2.102–2.262	4.41
ingest, one writer	31.699	16.810	16.663–17.330	1.89
ingest, four shards	32.549	18.805	18.644–18.879	1.73

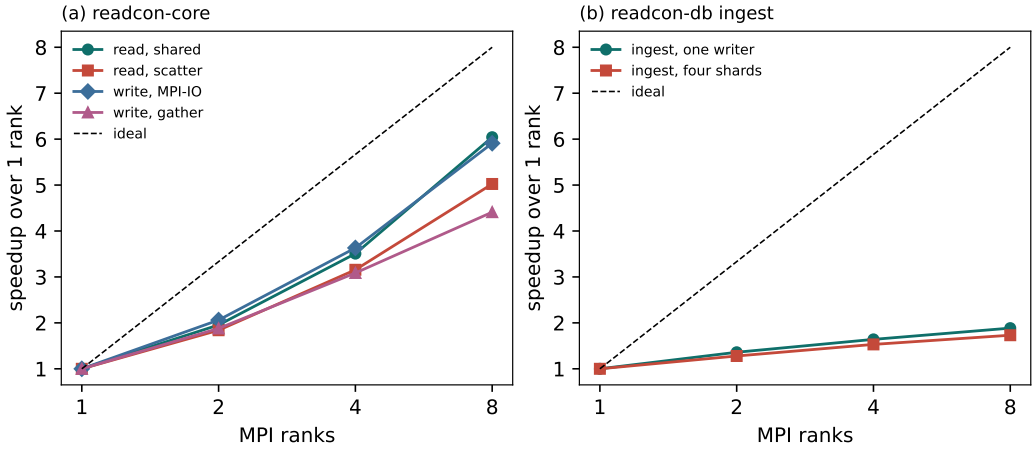


Figure 7: Strong-scaling speedup from the warm medians in Table 14. The dashed line is $t(1)/t(R) = R$. Ranks 1, 2, 4 and 8 on an 8-core host. Every output matched the serial bytes.

Weak scaling gives every rank the same frame count as the one-rank run. readcon-core holds 400,000 frames per rank, 135,600,000 bytes at one rank. readcon-db ingests 7,700 frames of 1,000 atoms per rank, four copies, 30,800 frames per rank. Table 15 is the warm medians, and the ratio is the eight-rank time over the one-rank time. Shared read grows by 1.36, scatter by 1.61,

MPI-IO write by 1.33, and gather write by 1.78. One-writer ingest grows by 3.77, from 7.841 s to 29.594 s. Four-shard ingest grows by 4.68, from 7.465 s to 34.899 s. The read and the MPI-IO write stay near the one-rank time. The ingest does not: eight ranks take several times as long as one rank on the same work per rank. Figure 8 is that growth. An earlier campaign with a different host and with means rather than medians is not this table.

Table 15: Weak scaling, warm page cache. Each rank holds the same frame count as the one-rank run. t_8/t_1 is the eight-rank median divided by the one-rank median. readcon-core holds 400,000 frames per rank (135,600,000 bytes at one rank). readcon-db ingests 7,700 frames of 1,000 atoms per rank, four copies (30,800 frames per rank). Medians of five repeats. Every output matches the serial bytes.

Path	1 rank (s)	8 ranks (s)	min–max at 8	t_8/t_1
read, shared file	1.161	1.581	1.573–1.792	1.36
read, scatter	1.218	1.962	1.949–2.264	1.61
write, MPI-IO	1.037	1.382	1.365–1.446	1.33
write, gather	1.211	2.161	2.097–2.296	1.78
ingest, one writer	7.841	29.594	28.943–30.364	3.77
ingest, four shards	7.465	34.899	34.807–36.533	4.68

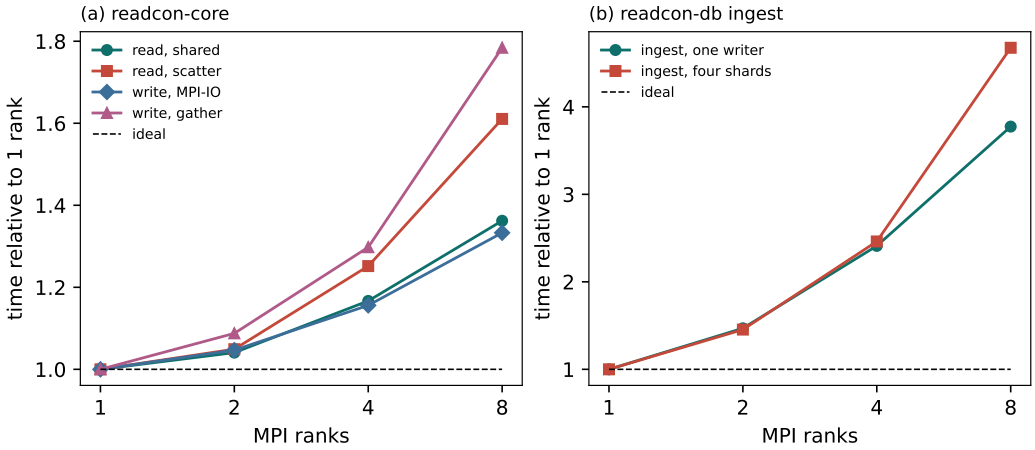


Figure 8: Weak scaling from the warm medians in Table 15. The dashed line is a ratio of 1, the same time as one rank. The ingest ratios lie above that line.

7.4. The store

The store measurements are the readcon-db evaluation at commit `6146c3b`, copied into `paper/cpc/data/db`. The host is an Intel Xeon, 8 CPUs, 47.07 GiB. The software is Python 3.12.3, rustc 1.88.0, readcon-db 0.1.8, readcon-core 0.16.2, ASE 3.29.0 and numpy 2.5.4. Threads are pinned to 1 so the three stores run under one cap. Five repeats are interleaved. Cold is a `posix_fadvise DONTNEED` hint and then a fresh process; the run did not drop the page cache. Warm is the next query in that process. The timed corpora have 1,000 and 10,000 frames, and every frame has its own xxHash3-128, so the ingest timing is not a deduplication win. The frames come from ASE EMT searches: an HCN/HNC band of 157 frames, a Cu adatom band of 159, a Pt–H band of 83, and a Pt adatom dimer of 71, plus an adaptive kinetic Monte Carlo trace of 4,000 steps on three systems. EMT on H, C and N is not a quantitative barrier. The HCN saddle in that trace is 197.6 eV with no hops, the Cu saddle is 5.23 eV with barrier about 1.09 eV and no hops, and the Pt–H trace has 1,664 hops and a barrier about 0.03 eV. eOn was not used for those searches. A 100,000-frame ingest was not timed: ASE's database writes about 200 frames/s and does not finish in two minutes. One million frames does not fit the default 2 GiB map.

At 10,000 frames the ingest medians are 0.311306 s for readcon-db (32,123 frames/s, 35.09 MB/s), 48.411 s for ASE's database committing every write (206.6 frames/s), and 5.985 s for one ASE connection that still commits every 5,000 rows (1,671 frames/s). The paired ratios are 0.006800 against the per-write commits and 0.05283 against the single connection. At 1,000 frames the readcon-db median is 0.03289 s (30,403 frames/s). The ingest table has 30 rows and the query table has 180, and every row passed the hit-count and index-sum check. The composition query is Cu with 9 atoms: 292 hits at 1,000 frames and 2,542 hits at 10,000 frames.

At 10,000 frames the readcon-db query medians are 8.14 ms, 10.1 ms and 9.57 ms cold for composition, energy and maximum force, and 0.176 ms, 0.211 ms and 0.208 ms warm. ASE's database takes 0.286 s, 0.223 s and 0.225 s cold, and 0.258 s, 0.203 s and 0.206 s warm. A directory of the same text in 64-frame chunks takes 0.193 s, 0.193 s and 0.197 s cold, and 0.140 s, 0.138 s and 0.138 s warm. The index is what the warm readcon-db number measures.

Allocated bytes at 10,000 frames are 10,923,745 for the raw CON text, 3,241,302 after gzip level 6, 3,312,800 after zstd level 3, 11,501,568 for the directory, 27,340,800 for readcon-db and 8,929,280 for ASE's database. readcon-db is 2.503 times the raw text: the text plus the indexes. The 2 GiB figure is

the map ceiling, not this file. At 1,000 frames zstd level 3 (270,863 bytes) is smaller than gzip level 6 (277,258 bytes). At 10,000 frames zstd is the larger of the two. The adaptive kinetic Monte Carlo trace has 24,000 frames and 27,864,917 stored blob bytes, equal to the raw file, and 7 unique hashes whose text is 8,319 bytes. A corpus of those seven texts allocates 98,304 bytes; the full trace allocates 74,911,744. The seven texts are the unhopped HCN state and its saddle, the unhopped Cu state and its saddle, and both Pt–H states with that saddle. Nine correctness checks pass: a round trip, identical bytes sharing one hash entry, a one-digit coordinate change, a missing force or velocity section, periodicity taken only from the declared flags, a rebuild of the indexes from the stored text, a second rebuild, an ignored numeric cache, and a hash entry that remains after a duplicate is rewritten. Section S10 records the repeats and the figures.

8. Limitations

A frame stores geometry and its labels. It does not store the short history of a quasi-Newton update or of a FIRE step. The control in Section 6 is the measurement: both printers agree with each other at 0.670723 eV and differ by 0.037113 eV from the uninterrupted barrier. The corpus decides the listed files. Agreement among the language bindings is agreement of one core, and the public Fortran wrapper shows that a binding can still fail while that core is unchanged. The resumed bands are one EMT Pt hop and one Morse band. They show what a lost field does to those searches. The EMT energies quoted for H, C and N are recorded as the trace produced them: the HCN saddle is 197.6 eV, and that potential is not a quantitative barrier for those atoms. Peer ratios hold for the library versions named in Sections 2 and 7.2. The MPI claim covers ranks 1 to 8 on the 8-core virtio host in Section 7.3. The default LMDB map is 2 GiB, and one million frames returns `MDB_MAP_FULL`. CON is the checkpoint record. H5MD is the trajectory record. The byte counts in Table 7 are the measured cost of the checkpoint text.

9. Conclusions

The geometry file in a rare-event campaign is the restart state of the search. The resume reads a per-axis mask, a persistent id, a declared time unit, binary64 coordinates with their periodic flags, and a version two readers agree on. On one masked slab, every common format tested here drops

at least one of those fields. The ASE POSCAR resume drops the height constraint and finishes 70.2 meV lower. Five extended XYZ readers return four answers for one file. CON 3.1 states the grammar, the corpus decides the listed files, and one parser serves the five languages. Twenty-five language pairs resume the climbing-image band at the uninterrupted barrier. The public Fortran wrapper, tested the same way, returns status -4 and writes no usable frame. A warm read costs 12.6 μs per 218-atom frame. Shared MPI reads and MPI-IO writes speed up across eight ranks on this host. The sharded ingest does not stay flat under weak scaling. readcon-db stores the CON text, indexes it, and ingests it in parallel; those are the operations Section 7.4 and Section 7.3 time.

Data availability

Version 0.17.0 of the reader is https://doi.org/10.5281/zenodo.23302640 [139]. The concept record is https://doi.org/10.5281/zenodo.18753567 [140]. Version 0.16.2, used by the store evaluation, is https://doi.org/10.5281/zenodo.23264985 [141]. The software citation points at those archived records [67, 64, 65, 66]. The store is archived separately [148]. Publishing a finished saddle remains the job of a repository, a materials archive or a reaction database, downstream of this checkpoint [111, 112, 113, 114, 115, 51]. The HCN coordinates are the public endpoints of that isomerisation [151, 152]. The specification, the grammar, the corpus and the campaign files are the supplementary material. The (s, d) figures use the projection defined for comparing bands that share atom identity [149].

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work received no funding.

Acknowledgements

The author thanks Hannes Jónsson for discussions on saddle searches and on what an eOn checkpoint has to carry, and Miha Gunde for discussions on matching atoms along a reaction path. Ruhila Goswami designed the colour palette of the figures. The library was written with the patience of the author's family.

References

[1] H. Eyring, The activated complex in chemical reactions, The Journal of Chemical Physics 3 (2) (1935) 107–115. doi:10.1063/1.1749604.
URL https://pubs.aip.org/jcp/article/3/2/107/203352/The-Activated-Complex-in-Chemical-Reactions

[2] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1-2) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.
URL https://linkinghub.elsevier.com/retrieve/pii/0022369757900598

[3] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/revmodphys.62.251.
URL https://link.aps.org/doi/10.1103/RevModPhys.62.251

[4] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: Classical and Quantum Dynamics in Condensed Phase Simulations, WORLD SCIENTIFIC, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.
URL http://www.worldscientific.com/doi/abs/10.1142/9789812839664_0016

[5] G. Henkelman, B. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904.

doi:10.1063/1.1329672.
URL https://pubs.aip.org/jcp/article/113/22/9901/185050/A-climbing-image-nudged-elastic-band-method-for

[6] G. Henkelman, H. Jónsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points, The Journal of Chemical Physics 113 (22) (2000) 9978–9985. doi:10.1063/1.1323224.
URL https://pubs.aip.org/jcp/article/113/22/9978/184858/Improved-tangent-estimate-in-the-nudged-elastic

[7] V. Ásgeirsson, B. O. Birgisson, R. Bjornsson, U. Becker, F. Neese, C. Riplinger, H. Jónsson, Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following, Journal of Chemical Theory and Computation 17 (8) (2021) 4929–4945. doi:10.1021/acs.jctc.1c00462.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.1c00462

[8] S. Smidstrup, A. Pedersen, K. Stokbro, H. Jónsson, Improved initial guess for minimum energy path calculations, The Journal of Chemical Physics 140 (21) (2014) 214106. doi:10.1063/1.4878664.
URL https://pubs.aip.org/jcp/article/140/21/214106/566679/Improved-initial-guess-for-minimum-energy-path

[9] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events, Physical Review B 66 (5) (2002) 052301. doi:10.1103/physrevb.66.052301.
URL https://link.aps.org/doi/10.1103/PhysRevB.66.052301

[10] W. E, W. Ren, E. Vanden-Eijnden, Simplified and improved string method for computing the minimum energy paths in barrier-crossing events, The Journal of Chemical Physics 126 (16) (2007) 164103. doi:10.1063/1.2720838.
URL https://pubs.aip.org/jcp/article/126/16/164103/905415/Simplified-and-improved-string-method-for

[11] B. Peters, A. Heyden, A. T. Bell, A. Chakraborty, A growing string method for determining transition states: Comparison to the nudged elastic band and string methods, The Journal of Chemical Physics 120 (17) (2004) 7877–7886. doi:10.1063/1.1691018.

URL https://pubs.aip.org/jcp/article/120/17/7877/534771/A-growing-string-method-for-determining-transition

[12] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.
URL https://pubs.aip.org/jcp/article/111/15/7010/475160/A-dimer-method-for-finding-saddle-points-on-high

[13] A. Heyden, A. T. Bell, F. J. Keil, Efficient methods for finding transition states in chemical reactions: Comparison of improved dimer method and partitioned rational function optimization method, The Journal of Chemical Physics 123 (22) (2005) 224101. doi:10.1063/1.2104507.
URL https://pubs.aip.org/jcp/article/123/22/224101/776452/Efficient-methods-for-finding-transition-states-in

[14] J. Kästner, P. Sherwood, Superlinearly converging dimer method for transition state search, The Journal of Chemical Physics 128 (1) (2008) 014106. doi:10.1063/1.2815812.
URL https://pubs.aip.org/jcp/article/128/1/014106/928095/Superlinearly-converging-dimer-method-for

[15] R. A. Olsen, G. J. Kroes, G. Henkelman, A. Arnaldsson, H. Jónsson, Comparison of methods for finding saddle points without knowledge of the final states, The Journal of Chemical Physics 121 (20) (2004) 9776–9792. doi:10.1063/1.1809574.
URL https://pubs.aip.org/jcp/article/121/20/9776/532635/Comparison-of-methods-for-finding-saddle-points

[16] D. J. Wales, Exploring energy landscapes, Annual Review of Physical Chemistry 69 (1) (2018) 401–425. doi:10.1146/annurev-physchem-050317-021219.
URL https://www.annualreviews.org/doi/10.1146/annurev-physchem-050317-021219

[17] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Accelerated saddle point refinement through full exploitation of partial Hessian diagonalization, Journal of Chemical Theory and Computation 15 (11) (2019)

6536–6549. doi:10.1021/acs.jctc.9b00869.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b00869

[18] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Geometry optimization speedup through a geodesic approach to internal coordinates, The Journal of Chemical Physics 155 (9) (2021) 094105. doi:10.1063/5.0060146.
URL https://pubs.aip.org/jcp/article/155/9/094105/199820/Geometry-optimization-speedup-through-a-geodesic

[19] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Sella, an open-source automation-friendly molecular saddle point optimizer, Journal of Chemical Theory and Computation 18 (11) (2022) 6974–6988. doi:10.1021/acs.jctc.2c00395.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.2c00395

[20] R. Goswami, M. Gunde, H. Jónsson, Enhanced climbing image nudged elastic band method with Hessian eigenmode alignment, Frontiers in Chemistry 14 (2026) 1807063. doi:10.3389/fchem.2026.1807063.
URL https://www.frontiersin.org/articles/10.3389/fchem.2026.1807063/full

[21] G. Henkelman, G. Jóhannesson, H. Jónsson, Methods for finding saddle points and minimum energy paths, in: Theoretical Methods in Condensed Phase Chemistry, Kluwer Academic Publishers, 2005, Ch. 10, pp. 269–302. doi:10.1007/0-306-46949-9_10.
URL http://link.springer.com/10.1007/0-306-46949-9_10

[22] A. F. Voter, Hyperdynamics: Accelerated molecular dynamics of infrequent events, Physical Review Letters 78 (20) (1997) 3908–3911. doi:10.1103/physrevlett.78.3908.
URL https://link.aps.org/doi/10.1103/PhysRevLett.78.3908

[23] R. A. Miron, K. A. Fichthorn, Accelerated molecular dynamics with the bond-boost method, The Journal of Chemical Physics 119 (12) (2003) 6210–6216. doi:10.1063/1.1603722.
URL https://pubs.aip.org/jcp/article/119/12/6210/451910/Accelerated-molecular-dynamics-with-the-bond-boost

[24] A. F. Voter, F. Montalenti, T. C. Germann, Extending the time scale in atomistic simulation of materials, Annual Review of Materials Research 32 (1) (2002) 321–346. doi:10.1146/annurev.matsci.32.112601.141541.
URL https://www.annualreviews.org/doi/10.1146/annurev.matsci.32.112601.141541

[25] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation and predefined event table, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.
URL https://pubs.aip.org/jcp/article/115/21/9657/453229/Long-time-scale-kinetic-Monte-Carlo-simulations

[26] A. F. Voter, Introduction to the kinetic Monte Carlo method, in: Radiation Effects in Solids, Springer Netherlands, 2007, Ch. 1, pp. 1–23. doi:10.1007/978-1-4020-5295-8_1.
URL http://link.springer.com/10.1007/978-1-4020-5295-8_1

[27] L. Xu, G. Henkelman, Adaptive kinetic Monte Carlo for first-principles accelerated dynamics, The Journal of Chemical Physics 129 (11) (2008) 114104. doi:10.1063/1.2976010.
URL https://pubs.aip.org/jcp/article/129/11/114104/296009/Adaptive-kinetic-Monte-Carlo-for-first-principles

[28] M. A. Novotny, Monte Carlo algorithms with absorbing Markov chains: Fast local algorithms for slow dynamics, Physical Review Letters 74 (1) (1995) 1–5. doi:10.1103/physrevlett.74.1.
URL https://link.aps.org/doi/10.1103/PhysRevLett.74.1

[29] A. F. Voter, Parallel replica method for dynamics of infrequent events, Physical Review B 57 (22) (1998) R13985–R13988. doi:10.1103/physrevb.57.r13985.
URL https://link.aps.org/doi/10.1103/PhysRevB.57.R13985

[30] D. Perez, B. P. Uberuaga, A. F. Voter, The parallel replica dynamics method – Coming of age, Computational Materials Science 100 (2015) 90–103. doi:10.1016/j.commatsci.2014.12.011.
URL https://linkinghub.elsevier.com/retrieve/pii/S0927025614008489

[31] D. Perez, E. D. Cubuk, A. Waterland, E. Kaxiras, A. F. Voter, Long-time dynamics through parallel trajectory splicing, Journal of Chemical Theory and Computation 12 (1) (2015) 18–28. doi:10.1021/acs.jctc.5b00916.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.5b00916

[32] D. J. Wales, J. P. K. Doye, Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms, The Journal of Physical Chemistry A 101 (28) (1997) 5111–5116. doi:10.1021/jp970984n.
URL https://pubs.acs.org/doi/10.1021/jp970984n

[33] S. Goedecker, Minima hopping: An efficient search method for the global minimum of the potential energy surface of complex molecular systems, The Journal of Chemical Physics 120 (21) (2004) 9911–9917. doi:10.1063/1.1724816.
URL https://pubs.aip.org/jcp/article/120/21/9911/533554/Minima-hopping-An-efficient-search-method-for-the

[34] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.
URL https://iopscience.iop.org/article/10.1088/0965-0393/22/5/055002

[35] R. Goswami, S. Chill, R. Terrell, G. Henkelman, M. Welborn, L. Zhang, A. Pedersen, E. Edelmann, J. C. Berthet, S. H. Jung, S. A. Ghasemi, A. Pena Torres, T. Dijkhuis, TheochemUI/eOn: v3.6.0, Zenodo, software, all versions (2026). doi:10.5281/zenodo.18529495.

[36] R. Goswami, S. Chill, R. Terrell, G. Henkelman, M. Welborn, L. Zhang, A. Pedersen, t brink, E. Edelmann, J. Claude, Alejandro, S. H. Jung, S. A. Ghasemi, chemist29, satishskamath, Maxim, T. Dijkhuis, via9a, TheochemUI/eOn: v3.6.0, Zenodo, software, version 3.6.0 (2026). doi:10.5281/zenodo.23166734.

[37] R. Goswami, Efficient exploration of chemical kinetics, arXiv:2510.21368 (2025). doi:10.48550/arxiv.2510.21368.

[38] A. Pedersen, H. Jónsson, Distributed implementation of the adaptive kinetic Monte Carlo method, Mathematics and Computers in Simulation 80 (7) (2010) 1487–1498. doi:10.1016/j.matcom.2009.02.010.
URL https://linkinghub.elsevier.com/retrieve/pii/S0378475409000500

[39] D. P. Anderson, BOINC: A system for public-resource computing and storage, in: Fifth IEEE/ACM International Workshop on Grid Computing, IEEE, 2004, pp. 4–10. doi:10.1109/grid.2004.14.
URL http://ieeexplore.ieee.org/document/1382809/

[40] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of Gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.5c00866

[41] R. Goswami, H. Jónsson, Adaptive pruning for increased robustness and reduced computational overhead in Gaussian process accelerated saddle point searches, ChemPhysChem 27 (4) (2026) e202500730. doi:10.1002/cphc.202500730.
URL https://chemistry-europe.onlinelibrary.wiley.com/doi/10.1002/cphc.202500730

[42] O.-P. Koistinen, V. Ásgeirsson, A. Vehtari, H. Jónsson, Nudged elastic band calculations accelerated with Gaussian process regression based on inverse interatomic distances, Journal of Chemical Theory and Computation 15 (12) (2019) 6738–6751. doi:10.1021/acs.jctc.9b00692.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b00692

[43] O.-P. Koistinen, V. Ásgeirsson, A. Vehtari, H. Jónsson, Minimum mode saddle point searches using Gaussian process regression with inverse-distance covariance function, Journal of Chemical Theory and Computation 16 (1) (2019) 499–509. doi:10.1021/acs.jctc.9b01038.
URL https://pubs.acs.org/doi/10.1021/acs.jctc.9b01038

[44] A. A. Peterson, Acceleration of saddle-point searches with machine learning, The Journal of Chemical Physics 145 (7) (2016) 074106. doi:

10.1063/1.4960708.
URL https://pubs.aip.org/jcp/article/145/7/074106/810392/Acceleration-of-saddle-point-searches-with-machine

[45] R. Goswami, A tutorial review of Bayesian optimization with Gaussian processes to accelerate stationary point searches, ACS Physical Chemistry Au 6 (4) (2026) 633–664. doi:10.1021/acsphyschemau.6c00038.
URL https://pubs.acs.org/doi/10.1021/acsphyschemau.6c00038

[46] F. Bigi, J. W. Abbott, P. Loche, A. Mazitov, D. Tisi, M. F. Langer, A. Goscinski, P. Pegolo, S. Chong, R. Goswami, P. Febrer, S. Chorna, M. Kellner, M. Ceriotti, G. Fraux, metatensor and metatomic: Foundational libraries for interoperable atomistic machine learning, The Journal of Chemical Physics 164 (6) (2026) 064113. doi:10.1063/5.0304911.
URL https://pubs.aip.org/jcp/article/164/6/064113/3379260/metatensor-and-metatomic-Foundational-libraries

[47] A. Mazitov, F. Bigi, M. Kellner, P. Pegolo, D. Tisi, G. Fraux, S. Pozdnyakov, P. Loche, M. Ceriotti, PET-MAD as a lightweight universal interatomic potential for advanced materials modeling, Nature Communications 16 (1) (2025) 10653. doi:10.1038/s41467-025-65662-7.
URL https://www.nature.com/articles/s41467-025-65662-7

[48] L. Himanen, M. O. Jäger, E. V. Morooka, F. F. Canova, Y. S. Ranawat, D. Z. Gao, P. Rinke, A. S. Foster, DScribe: Library of descriptors for machine learning in materials science, Computer Physics Communications 247 (2020) 106949. doi:10.1016/j.cpc.2019.106949.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465519303042

[49] A. Togo, K. Shinohara, I. Tanaka, Spglib: a software library for crystal symmetry search, Science and Technology of Advanced Materials: Methods 4 (1) (2024) 2384822. doi:10.1080/27660400.2024.2384822.
URL https://www.tandfonline.com/doi/full/10.1080/27660400.2024.2384822

[50] S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, G. Ceder, Python Materials

Genomics (pymatgen): A robust, open-source Python library for materials analysis, Computational Materials Science 68 (2013) 314–319. doi:10.1016/j.commatsci.2012.10.028.
URL https://linkinghub.elsevier.com/retrieve/pii/S0927025612006295

[51] S. P. Ong, S. Cholia, A. Jain, M. Brafman, D. Gunter, G. Ceder, K. A. Persson, The Materials Application Programming Interface (API): A simple, flexible and efficient API for materials data based on REpresentational State Transfer (REST) principles, Computational Materials Science 97 (2015) 209–215. doi:10.1016/j.commatsci.2014.10.037.
URL https://linkinghub.elsevier.com/retrieve/pii/S0927025614007113

[52] H. Jónsson, Simulation of surface processes, Proceedings of the National Academy of Sciences 108 (3) (2011) 944–949. doi:10.1073/pnas.1006670108.
URL https://pnas.org/doi/full/10.1073/pnas.1006670108

[53] M. Gunde, N. Salles, A. Hémeryck, L. Martin-Samos, IRA: A shape matching approach for recognition and comparison of generic atomic patterns, Journal of Chemical Information and Modeling 61 (11) (2021) 5446–5457. doi:10.1021/acs.jcim.1c00567.
URL https://pubs.acs.org/doi/10.1021/acs.jcim.1c00567

[54] R. Terrell, M. Welborn, S. T. Chill, G. Henkelman, Database of atomistic reaction mechanisms with application to kinetic Monte Carlo, The Journal of Chemical Physics 137 (1) (2012) 014105. doi:10.1063/1.4730746.
URL https://pubs.aip.org/jcp/article/137/1/014105/191477/Database-of-atomistic-reaction-mechanisms-with

[55] E. Tiesinga, P. J. Mohr, D. B. Newell, B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, Reviews of Modern Physics 93 (2) (2021) 025010. doi:10.1103/revmodphys.93.025010.
URL https://link.aps.org/doi/10.1103/RevModPhys.93.025010

[56] S. McKeever, O. Bennich-Björkman, O.-A. Salah, Unit of measurement libraries, their popularity and suitability, Software: Practice and Expe-

rience 51 (4) (2020) 711–734. doi:10.1002/spe.2926.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.2926

[57] W. C. Swope, H. C. Andersen, P. H. Berens, K. R. Wilson, A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters, The Journal of Chemical Physics 76 (1) (1982) 637–649. doi:10.1063/1.442716.
URL https://pubs.aip.org/jcp/article/76/1/637/397800/A-computer-simulation-method-for-the-calculation

[58] G. Bussi, D. Donadio, M. Parrinello, Canonical sampling through velocity rescaling, The Journal of Chemical Physics 126 (1) (2007) 014101. doi:10.1063/1.2408420.
URL https://pubs.aip.org/jcp/article/126/1/014101/186581/Canonical-sampling-through-velocity-rescaling

[59] R. Gowers, M. Linke, J. Barnoud, T. Reddy, M. Melo, S. Seyler, J. Domański, D. Dotson, S. Buchoux, I. Kenney, O. Beckstein, MDAnalysis: A Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the Python in Science Conference, SciPy, 2016, pp. 98–105. doi:10.25080/majora-629e541a-00e.
URL https://doi.curvenote.com/10.25080/Majora-629e541a-00e

[60] R. D. Peng, Reproducible research in computational science, Science 334 (6060) (2011) 1226–1227. doi:10.1126/science.1213847.
URL https://www.science.org/doi/10.1126/science.1213847

[61] D. C. Ince, L. Hatton, J. Graham-Cumming, The case for open computer programs, Nature 482 (7386) (2012) 485–488. doi:10.1038/nature10836.
URL https://www.nature.com/articles/nature10836

[62] K. Hinsen, Caring for your data, Computing in Science & Engineering 14 (6) (2012) 70–74. doi:10.1109/mcse.2012.108.
URL http://ieeexplore.ieee.org/document/6341744/

[63] K. Hinsen, Dealing with software collapse, Computing in Science & Engineering 21 (3) (2019) 104–108. doi:10.1109/mcse.2019.2900945.
URL https://ieeexplore.ieee.org/document/8701540/

[64] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR Guiding Principles for scientific data management and stewardship, Scientific Data 3 (1) (2016) 160018. doi:10.1038/sdata.2016.18.
URL https://www.nature.com/articles/sdata201618

[65] M. Barker, N. P. Chue Hong, D. S. Katz, A.-L. Lamprecht, C. Martinez-Ortiz, F. Psomopoulos, J. Harrow, L. J. Castro, M. Gruenpeter, P. A. Martinez, T. Honeyman, Introducing the FAIR Principles for research software, Scientific Data 9 (1) (2022) 622. doi:10.1038/s41597-022-01710-x.
URL https://www.nature.com/articles/s41597-022-01710-x

[66] A.-L. Lamprecht, L. Garcia, M. Kuzak, C. Martinez, R. Arcila, E. M. D. Pico, V. D. D. Angel, S. v. d. Sandt, J. Ison, P. A. Martinez, P. McQuilton, A. Valencia, J. Harrow, F. Psomopoulos, J. L. Gelpi, N. C. Hong, C. Goble, S. Capella-Gutierrez, Towards FAIR principles for research software, Data Science 3 (1) (2019) 37–59. doi:10.3233/ds-190026.
URL https://journals.sagepub.com/doi/10.3233/DS-190026

[67] A. M. Smith, D. S. Katz, K. E. Niemeyer, F. S. C. W. Group, Software citation principles, PeerJ Computer Science 2 (2016) e86. doi:10.7717/peerj-cs.86.
URL https://peerj.com/articles/cs-86

[68] P. Murray-Rust, H. S. Rzepa, Chemical markup, XML, and the worldwide web. 1. basic principles, Journal of Chemical Information and Computer Sciences 39 (6) (1999) 928–942. doi:10.1021/ci990052b.
URL https://pubs.acs.org/doi/10.1021/ci990052b

[69] K. Hinsen, MOSAIC: A data model and file formats for molecular simulations, Journal of Chemical Information and Modeling 54 (1) (2013) 131–137. doi:10.1021/ci400599y.
URL https://pubs.acs.org/doi/10.1021/ci400599y

[70] S. R. Hall, F. H. Allen, I. D. Brown, The crystallographic information file (CIF): a new standard archive file for crystallography, Acta Crystallographica Section A Foundations of Crystallography 47 (6) (1991) 655–685. doi:10.1107/s010876739101067x.
URL https://journals.iucr.org/paper?S010876739101067X

[71] A. L. Spek, checkCIF validation ALERTS: what they mean and how to respond, Acta Crystallographica Section E Crystallographic Communications 76 (1) (2020) 1–11. doi:10.1107/s2056989019016244.
URL https://journals.iucr.org/paper?S2056989019016244

[72] J. Nocedal, Updating quasi-Newton matrices with limited storage, Mathematics of Computation 35 (151) (1980) 773–782. doi:10.1090/s0025-5718-1980-0572855-7.
URL https://www.ams.org/mcom/1980-35-151/S0025-5718-1980-0572855-7/

[73] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, The Journal of Chemical Physics 128 (13) (2008) 134106. doi:10.1063/1.2841941.
URL https://pubs.aip.org/jcp/article/128/13/134106/977389/Optimization-methods-for-finding-minimum-energy

[74] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, Structural relaxation made simple, Physical Review Letters 97 (17) (2006) 170201. doi:10.1103/physrevlett.97.170201.
URL https://link.aps.org/doi/10.1103/PhysRevLett.97.170201

[75] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz,

O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648x/aa680e.
URL https://iopscience.iop.org/article/10.1088/1361-648X/aa680e

[76] S. R. Bahn, K. W. Jacobsen, An object-oriented scripting interface to a legacy electronic structure code, Computing in Science & Engineering 4 (3) (2002) 56–66. doi:10.1109/5992.998641.
URL http://ieeexplore.ieee.org/document/998641/

[77] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, Pelsa, M. Levesque, S. Maiti, S. Guionniere, Jmintser, chemfiles/chemfiles: Version 0.10.4, Zenodo, software, version 0.10.4 (2023). doi:10.5281/zenodo.7904565.

[78] R. Gowers, O. Beckstein, M. Linke, I. Alibay, J. Barnoud, T. Reddy, H. MacDermott-Opeskin, R. Meli, L. Wang, U. Bansal, J. Zeman, J. Detlefs, D. L. Dotson, M. N. Melo, M. Bieniek, ayushsuhane, S. Buchoux, M. Tiberti, N. Michaud-Agrawal, D. Cruz, S. Seyler, denniej0 2, P. Loche, J. Borreguero, Y. Zhuang, T. Yelgoe, N. Pal, Ninad, J. Domański, Z. Wu, MDAnalysis/mdanalysis: Release 2.10.0 of MDAnalysis and MDAnalysisTests, Zenodo, software, version package-2.10.0 (2025). doi:10.5281/zenodo.17382469.

[79] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.21787

[80] A. Collette, T. Kluyver, T. A. Caswell, J. Tocknell, J. Kieffer, A. Jelenak, A. Scopatz, D. Dale, Chen, T. VINCENT, M. Einhorn, Payno, Juliagarriga, P. Sciarelli, V. Valls, S. Ghosh, U. K. Pedersen, Jakirkham, M. Raspaud, C. Danilevski, H. Abbasi, J. Readey, K. Mühlbauer, A. Paramonov, L. Chan, V. A. Solé, Jialin, D. H. Guest, Y. Feng, M. Kittisopikul, h5py/h5py: 3.7.0, Zenodo, software, version 3.7.0 (2022). doi:10.5281/zenodo.6575970.

[81] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F.-X. Coudert, M. Levesque, S. Maiti, S. Guionniere, chemfiles/chemfiles: Version 0.10.4, Zenodo, software, all versions (2023). doi:10.5281/zenodo.1202207.

[82] G. Kresse, J. Furthmüller, Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set, Physical Review B 54 (16) (1996) 11169–11186. doi:10.1103/physrevb.54.11169.
URL https://link.aps.org/doi/10.1103/PhysRevB.54.11169

[83] G. Kresse, J. Furthmüller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set, Computational Materials Science 6 (1) (1996) 15–50. doi:10.1016/0927-0256(96)00008-0.
URL https://linkinghub.elsevier.com/retrieve/pii/0927025696000080

[84] P. d. Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465514000447

[85] M. Folk, G. Heber, Q. Koziol, E. Pourmal, D. Robinson, An overview of the HDF5 technology suite and its applications, in: Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases, ACM, 2011, pp. 36–47. doi:10.1145/1966895.1966900.
URL https://dl.acm.org/doi/10.1145/1966895.1966900

[86] D. Spångberg, D. S. D. Larsson, D. v. d. Spoel, Trajectory NG: portable, compressed, general molecular dynamics trajectories, Journal of Molecular Modeling 17 (10) (2011) 2669–2685. doi:10.1007/s00894-010-0948-5.
URL http://link.springer.com/10.1007/s00894-010-0948-5

[87] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2013) 260–269. doi:10.1002/jcc.

23495.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.23495

[88] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1-2 (2015) 19–25. doi:10.1016/j.softx.2015.06.001.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352711015000059

[89] B. R. Brooks, C. L. Brooks, A. D. Mackerell, L. Nilsson, R. J. Petrella, B. Roux, Y. Won, G. Archontis, C. Bartels, S. Boresch, A. Caflisch, L. Caves, Q. Cui, A. R. Dinner, M. Feig, S. Fischer, J. Gao, M. Hodoscek, W. Im, K. Kuczera, T. Lazaridis, J. Ma, V. Ovchinnikov, E. Paci, R. W. Pastor, C. B. Post, J. Z. Pu, M. Schaefer, B. Tidor, R. M. Venable, H. L. Woodcock, X. Wu, W. Yang, D. M. York, M. Karplus, CHARMM: The biomolecular simulation program, Journal of Computational Chemistry 30 (10) (2009) 1545–1614. doi:10.1002/jcc.21287.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.21287

[90] D. A. Case, T. E. Cheatham, T. Darden, H. Gohlke, R. Luo, K. M. Merz, A. Onufriev, C. Simmerling, B. Wang, R. J. Woods, The Amber biomolecular simulation programs, Journal of Computational Chemistry 26 (16) (2005) 1668–1688. doi:10.1002/jcc.20290.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.20290

[91] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.
URL http://ieeexplore.ieee.org/document/56302/

[92] S. A. Brown, M. Folk, G. Goucher, R. Rew, P. F. Dubois, Software for portable scientific data management, Computers in Physics 7 (3) (1993) 304–308. doi:10.1063/1.4823180.
URL https://pubs.aip.org/cip/article/7/3/304/281673/Software-for-Portable-Scientific-Data-Management

[93] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, V. S. Pande, MDTraj: A modern open library for the analysis of molecular

dynamics trajectories, Biophysical Journal 109 (8) (2015) 1528–1532. doi:10.1016/j.bpj.2015.08.015.
URL https://linkinghub.elsevier.com/retrieve/pii/S0006349515008267

[94] W. Humphrey, A. Dalke, K. Schulten, VMD: Visual molecular dynamics, Journal of Molecular Graphics 14 (1) (1996) 33–38. doi:10.1016/0263-7855(96)00018-5.
URL https://linkinghub.elsevier.com/retrieve/pii/0263785596000185

[95] A. Stukowski, Visualization and analysis of atomistic simulation data with OVITO–the Open Visualization Tool, Modelling and Simulation in Materials Science and Engineering 18 (1) (2009) 015012. doi:10.1088/0965-0393/18/1/015012.
URL https://iopscience.iop.org/article/10.1088/0965-0393/18/1/015012

[96] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck, A. Huebl, M. Kim, J. Kress, T. Kurc, Q. Liu, J. Logan, K. Mehta, G. Ostrouchov, M. Parashar, F. Poeschel, D. Pugmire, E. Suchyta, K. Takahashi, N. Thompson, S. Tsutsumi, L. Wan, M. Wolf, K. Wu, S. Klasky, ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management, SoftwareX 12 (2020) 100561. doi:10.1016/j.softx.2020.100561.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352711019302560

[97] T. Meyer, C. Ferrer-Costa, A. Pérez, M. Rueda, A. Bidon-Chanal, F. J. Luque, C. A. Laughton, M. Orozco, Essential dynamics: A tool for efficient trajectory compression and management, Journal of Chemical Theory and Computation 2 (2) (2006) 251–258. doi:10.1021/ct050285b.
URL https://pubs.acs.org/doi/10.1021/ct050285b

[98] R. Goswami, A. Goswami, J. K. Singh, d-SEAMS: Deferred structural elucidation analysis for molecular simulations, Journal of Chemical Information and Modeling 60 (4) (2020) 2169–2177. doi:10.1021/acs.

jcim.0c00031.
URL https://pubs.acs.org/doi/10.1021/acs.jcim.0c00031

[99] E. Jakupovic, O. Beckstein, MPI-parallel molecular dynamics trajectory analysis with the H5MD format in the MDAnalysis Python package, in: Proceedings of the Python in Science Conference, SciPy, 2021, pp. 40–48. doi:10.25080/majora-1b6fd038-005.
URL https://doi.curvenote.com/10.25080/majora-1b6fd038-005

[100] Open Geospatial Consortium, Zarr storage specification 2.0 community standard (2022). doi:10.62973/21-050r1.
URL https://docs.ogc.org/doi/21-050r1

[101] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, P. E. Bourne, The protein data bank, Nucleic Acids Research 28 (1) (2000) 235–242. doi:10.1093/nar/28.1.235.
URL https://academic.oup.com/nar/article-lookup/doi/10.1093/nar/28.1.235

[102] H. Berman, K. Henrick, H. Nakamura, Announcing the worldwide protein data bank, Nature Structural & Molecular Biology 10 (12) (2003) 980–980. doi:10.1038/nsb1203-980.
URL https://www.nature.com/articles/nsb1203-980

[103] J. D. Westbrook, P. E. Bourne, STAR/mmCIF: An ontology for macromolecular structure, Bioinformatics 16 (2) (2000) 159–168. doi:10.1093/bioinformatics/16.2.159.
URL https://academic.oup.com/bioinformatics/article/16/2/159/232068

[104] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465521002836

[105] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files, Computer Physics Communications 197 (2015) 212–219. doi:10.1016/j.cpc.2015.07.012.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465515002817

[106] N. M. O'Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, G. R. Hutchison, Open Babel: An open chemical toolbox, Journal of Cheminformatics 3 (1) (2011) 33. doi:10.1186/1758-2946-3-33.
URL https://jcheminf.biomedcentral.com/articles/10.1186/1758-2946-3-33

[107] N. M. O'boyle, A. L. Tenderholt, K. M. Langner, cclib: A library for package-independent computational chemistry algorithms, Journal of Computational Chemistry 29 (5) (2007) 839–845. doi:10.1002/jcc.20823.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.20823

[108] T. Verstraelen, W. Adams, L. Pujal, A. Tehrani, B. D. Kelly, L. Macaya, F. Meng, M. Richer, R. Hernández-Esparza, X. D. Yang, M. Chan, T. D. Kim, M. Cools-Ceuppens, V. Chuiko, E. Vöhringer-Martinez, P. W. Ayers, F. Heidar-Zadeh, IOData: A Python library for reading, writing, and converting computational chemistry file formats and generating input files, Journal of Computational Chemistry 42 (6) (2020) 458–464. doi:10.1002/jcc.26468.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.26468

[109] E. Posenitskiy, V. G. Chilkuri, A. Ammar, M. Hapka, K. Pernal, R. Shinde, E. J. L. Borda, C. Filippi, K. Nakano, O. Kohulák, S. Sorella, P. d. O. Castro, W. Jalby, P. L. Ríos, A. Alavi, A. Scemama, TREXIO: A file format and library for quantum chemistry, The Journal of Chemical Physics 158 (17) (2023) 174801. doi:10.1063/5.0148161.
URL https://pubs.aip.org/jcp/article/158/17/174801/2888846/TREXIO-A-file-format-and-library-for-quantum

[110] D. G. A. Smith, D. Altarawy, L. A. Burns, M. Welborn, L. N. Naden, L. Ward, S. Ellis, B. P. Pritchard, T. D. Crawford, The MolSSI QCA rchive project: An open-source platform to compute, organize, and share quantum chemistry data, WIREs Computational Molecular Science

11 (2) (2020) e1491. doi:10.1002/wcms.1491.
URL https://wires.onlinelibrary.wiley.com/doi/10.1002/wcms.1491

[111] C. W. Andersen, R. Armiento, E. Blokhin, G. J. Conduit, S. Dwaraknath, M. L. Evans, Á. Fekete, A. Gopakumar, S. Gražulis, A. Merkys, F. Mohamed, C. Oses, G. Pizzi, G.-M. Rignanese, M. Scheidgen, L. Talirz, C. Toher, D. Winston, R. Aversa, K. Choudhary, P. Colinet, S. Curtarolo, D. Di Stefano, C. Draxl, S. Er, M. Esters, M. Fornari, M. Giantomassi, M. Govoni, G. Hautier, V. Hegde, M. K. Horton, P. Huck, G. Huhs, J. Hummelshøj, A. Kariryaa, B. Kozinsky, S. Kumbhar, M. Liu, N. Marzari, A. J. Morris, A. A. Mostofi, K. A. Persson, G. Petretto, T. Purcell, F. Ricci, F. Rose, M. Scheffler, D. Speckhard, M. Uhrin, A. Vaitkus, P. Villars, D. Waroquiers, C. Wolverton, M. Wu, X. Yang, OPTIMADE, an API for exchanging materials data, Scientific Data 8 (1) (2021) 217. doi:10.1038/s41597-021-00974-z.
URL https://www.nature.com/articles/s41597-021-00974-z

[112] C. Draxl, M. Scheffler, The NOMAD laboratory: from data sharing to artificial intelligence, Journal of Physics: Materials 2 (3) (2019) 036001. doi:10.1088/2515-7639/ab13bb.
URL https://iopscience.iop.org/article/10.1088/2515-7639/ab13bb

[113] K. T. Winther, M. J. Hoffmann, J. R. Boes, O. Mamun, M. Bajdich, T. Bligaard, Catalysis-Hub.org, an open electronic structure database for surface reactions, Scientific Data 6 (1) (2019) 75. doi:10.1038/s41597-019-0081-y.
URL https://www.nature.com/articles/s41597-019-0081-y

[114] M. Schreiner, A. Bhowmik, T. Vegge, J. Busk, O. Winther, Transition1x – a dataset for building generalizable reactive machine learning potentials, Scientific Data 9 (1) (2022) 779. doi:10.1038/s41597-022-01870-w.
URL https://www.nature.com/articles/s41597-022-01870-w

[115] L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu, A. Palizhati, A. Sriram, B. Wood, J. Yoon, D. Parikh, C. L. Zitnick, Z. Ulissi, Open catalyst 2020 (OC20) dataset and community challenges, ACS Catalysis 11 (10)

(2021) 6059–6072. doi:10.1021/acscatal.0c04525.
URL https://pubs.acs.org/doi/10.1021/acscatal.0c04525

[116] L. Martínez, R. Andrade, E. G. Birgin, J. M. Martínez, PACKMOL: A package for building initial configurations for molecular dynamics simulations, Journal of Computational Chemistry 30 (13) (2009) 2157–2164. doi:10.1002/jcc.21224.
URL https://onlinelibrary.wiley.com/doi/10.1002/jcc.21224

[117] G. A. Tribello, M. Bonomi, D. Branduardi, C. Camilloni, G. Bussi, PLUMED 2: New feathers for an old bird, Computer Physics Communications 185 (2) (2014) 604–613. doi:10.1016/j.cpc.2013.09.018.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465513003196

[118] F. Weik, R. Weeber, K. Szuttor, K. Breitsprecher, J. d. Graaf, M. Kuron, J. Landsgesell, H. Menke, D. Sean, C. Holm, ESPResSo 4.0 – an extensible software package for simulating soft matter systems, The European Physical Journal Special Topics 227 (14) (2019) 1789–1816. doi:10.1140/epjst/e2019-800186-9.
URL http://link.springer.com/10.1140/epjst/e2019-800186-9

[119] M. Ceriotti, J. More, D. E. Manolopoulos, i-PI: A Python interface for ab initio path integral molecular dynamics simulations, Computer Physics Communications 185 (3) (2014) 1019–1026. doi:10.1016/j.cpc.2013.10.027.
URL https://linkinghub.elsevier.com/retrieve/pii/S001046551300372X

[120] V. Kapil, M. Rossi, O. Marsalek, R. Petraglia, Y. Litman, T. Spura, B. Cheng, A. Cuzzocrea, R. H. Meißner, D. M. Wilkins, B. A. Helfrecht, P. Juda, S. P. Bienvenue, W. Fang, J. Kessler, I. Poltavsky, S. Vandenbrande, J. Wieme, C. Corminboeuf, T. D. Kühne, D. E. Manolopoulos, T. E. Markland, J. O. Richardson, A. Tkatchenko, G. A. Tribello, V. Van Speybroeck, M. Ceriotti, i-PI 2.0: A universal force engine for advanced molecular simulations, Computer Physics Communications 236 (2019) 214–223. doi:10.1016/j.cpc.2018.09.020.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465518303436

[121] Y. Litman, V. Kapil, Y. M. Y. Feldman, D. Tisi, T. Begušić, K. Fidanyan, G. Fraux, J. Higer, M. Kellner, T. E. Li, E. S. Pós, E. Stocco, G. Trenins, B. Hirshberg, M. Rossi, M. Ceriotti, i-PI 3.0: A flexible and efficient framework for advanced atomistic simulations, The Journal of Chemical Physics 161 (6) (2024) 062504. doi:10.1063/5.0215869.
URL https://pubs.aip.org/jcp/article/161/6/062504/3308034/i-PI-3-0-A-flexible-and-efficient-framework-for

[122] T. A. Barnes, E. Marin-Rimoldi, S. Ellis, T. D. Crawford, The MolSSI Driver Interface Project: A framework for standardized, on-the-fly interoperability between computational molecular sciences codes, Computer Physics Communications 261 (2021) 107688. doi:10.1016/j.cpc.2020.107688.
URL https://linkinghub.elsevier.com/retrieve/pii/S0010465520303386

[123] M. J. T. Oliveira, N. Papior, Y. Pouillon, V. Blum, E. Artacho, D. Caliste, F. Corsetti, S. d. Gironcoli, A. M. Elena, A. García, V. M. García-Suárez, L. Genovese, W. P. Huhn, G. Huhs, S. Kokott, E. Küçükbenli, A. H. Larsen, A. Lazzaro, I. V. Lebedeva, Y. Li, D. López-Durán, P. López-Tarifa, M. Lüders, M. A. L. Marques, J. Minar, S. Mohr, A. A. Mostofi, A. O'Cais, M. C. Payne, T. Ruh, D. G. A. Smith, J. M. Soler, D. A. Strubbe, N. Tancogne-Dejean, D. Tildesley, M. Torrent, V. W.-z. Yu, The CECAM electronic structure library and the modular software development paradigm, The Journal of Chemical Physics 153 (2) (2020) 024117. doi:10.1063/5.0012901.
URL https://pubs.aip.org/jcp/article/153/2/024117/1061500/The-CECAM-electronic-structure-library-and-the

[124] T. Bray, The JavaScript object notation (JSON) data interchange format, Tech. Rep. 8259, IETF (2017). doi:10.17487/rfc8259.
URL https://www.rfc-editor.org/info/rfc8259

[125] G. Langdale, D. Lemire, Parsing gigabytes of JSON per second, The VLDB Journal 28 (6) (2019) 941–960. doi:10.1007/s00778-019-00578-5.
URL http://link.springer.com/10.1007/s00778-019-00578-5

[126] H. Bekker, Unification of box shapes in molecular simulations, Journal of Computational Chemistry 18 (15) (1997) 1930–1942. doi:10.1002/

(sici)1096-987x(19971130)18:15<1930::aid-jcc8>3.0.co;2-p.
URL https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1096-987X(19971130)18:15<1930::AID-JCC8>3.0.CO;2-P

[127] IEEE Computer Society, IEEE standard for floating-point arithmetic (2019). doi:10.1109/ieeestd.2019.8766229.
URL https://ieeexplore.ieee.org/document/8766229/

[128] D. Goldberg, What every computer scientist should know about floating-point arithmetic, ACM Computing Surveys 23 (1) (1991) 5–48. doi:10.1145/103162.103163.
URL https://dl.acm.org/doi/10.1145/103162.103163

[129] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 92–101. doi:10.1145/93542.93557.
URL http://portal.acm.org/citation.cfm?doid=93542.93557

[130] W. D. Clinger, How to read floating point numbers accurately, ACM SIGPLAN Notices 39 (4) (2004) 360–371. doi:10.1145/989393.989430.
URL https://dl.acm.org/doi/10.1145/989393.989430

[131] G. L. Steele, J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 112–126. doi:10.1145/93542.93559.
URL http://portal.acm.org/citation.cfm?doid=93542.93559

[132] G. L. Steele, J. L. White, How to print floating-point numbers accurately, ACM SIGPLAN Notices 39 (4) (2004) 372–389. doi:10.1145/989393.989431.
URL https://dl.acm.org/doi/10.1145/989393.989431

[133] R. G. Burger, R. K. Dybvig, Printing floating-point numbers quickly and accurately, in: Proceedings of the ACM SIGPLAN 1996 conference on Programming language design and implementation, ACM, 1996, pp. 108–116. doi:10.1145/231379.231397.
URL https://dl.acm.org/doi/10.1145/231379.231397

[134] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, in: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2010, pp. 233–243. doi:10.1145/1806596.1806623.
URL https://dl.acm.org/doi/10.1145/1806596.1806623

[135] M. Andrysco, R. Jhala, S. Lerner, Printing floating-point numbers: a faster, always correct method, in: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM, 2016, pp. 555–567. doi:10.1145/2837614.2837654.
URL https://dl.acm.org/doi/10.1145/2837614.2837654

[136] U. Adams, Ryū: fast float-to-string conversion, in: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2018, pp. 270–282. doi:10.1145/3192366.3192369.
URL https://dl.acm.org/doi/10.1145/3192366.3192369

[137] P. Deutsch, GZIP file format specification version 4.3, Tech. Rep. 1952, IETF (1996). doi:10.17487/rfc1952.
URL https://www.rfc-editor.org/info/rfc1952

[138] Y. Collet, Zstandard compression and the 'application/zstd' media type, Tech. Rep. 8878, IETF (2021). doi:10.17487/rfc8878.
URL https://www.rfc-editor.org/info/rfc8878

[139] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, version 0.17.0 (2026). doi:10.5281/zenodo.23302640.

[140] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, all versions (2026). doi:10.5281/zenodo.18753567.

[141] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, version 0.16.2 (2026). doi:10.5281/zenodo.23264985.

[142] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith,

R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.
URL https://www.nature.com/articles/s41586-020-2649-2

[143] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.
URL https://epubs.siam.org/doi/10.1137/141000671

[144] R. Jung, J.-H. Jourdan, R. Krebbers, D. Dreyer, RustBelt: securing the foundations of the Rust programming language, Proceedings of the ACM on Programming Languages 2 (POPL) (2017) 1–34. doi:10.1145/3158154.
URL https://dl.acm.org/doi/10.1145/3158154

[145] J. M. Perkel, Why scientists are turning to Rust, Nature 588 (7836) (2020) 185–186. doi:10.1038/d41586-020-03382-2.
URL https://www.nature.com/articles/d41586-020-03382-2

[146] D. Lemire, Number parsing at a gigabyte per second, Software: Practice and Experience 51 (8) (2021) 1700–1727. doi:10.1002/spe.2984.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.2984

[147] N. Mushtak, D. Lemire, Fast number parsing without fallback, Software: Practice and Experience 53 (7) (2023) 1467–1471. doi:10.1002/spe.3198.
URL https://onlinelibrary.wiley.com/doi/10.1002/spe.3198

[148] R. Goswami, readcon-db: mmap CON corpus store with multi-language bindings, https://github.com/lode-org/readcon-db, software, version 0.1.8 (2026).

[149] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126000683

[150] G. L. Kellogg, P. J. Feibelman, Surface self-diffusion on Pt(001) by an atomic exchange mechanism, Physical Review Letters 64 (26) (1990) 3143–3146. doi:10.1103/physrevlett.64.3143.
URL https://link.aps.org/doi/10.1103/PhysRevLett.64.3143

[151] T. van Mourik, G. J. Harris, O. L. Polyansky, J. Tennyson, A. G. Császár, P. J. Knowles, Ab initio global potential, dipole, adiabatic, and relativistic correction surfaces for the HCN–HNC system, The Journal of Chemical Physics 115 (8) (2001) 3706–3718. doi:10.1063/1.1383586.
URL https://pubs.aip.org/jcp/article/115/8/3706/463154/Ab-initio-global-potential-dipole-adiabatic-and

[152] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.
URL https://linkinghub.elsevier.com/retrieve/pii/S2215016126001172

[153] B. Efron, Better bootstrap confidence intervals, Journal of the American Statistical Association 82 (397) (1987) 171–185. doi:10.1080/01621459.1987.10478410.
URL http://www.tandfonline.com/doi/abs/10.1080/01621459.1987.10478410

[154] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th international conference on Architectural support for programming languages and operating systems, ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.
URL https://dl.acm.org/doi/10.1145/1508244.1508275

[155] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 international symposium on memory management, ACM, 2013, pp. 63–74. doi:10.1145/2491894.2464160.
URL https://dl.acm.org/doi/10.1145/2491894.2464160

[156] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, ACM,

2015, pp. 1–12. doi:10.1145/2807591.2807644.
URL https://dl.acm.org/doi/10.1145/2807591.2807644

[157] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.